



Detecting AI-generated text in high-resource languages: developing a RoBERTa-CNN hybrid model for academic integrity challenge

Manish Prajapati¹ · Santos Kumar Baliarsingh¹ · Prabhu Prasad Dev¹

Received: 4 February 2025 / Accepted: 11 November 2025 / Published online: 29 January 2026
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2026

Abstract

With the emergence of ChatGPT, an advanced generative AI tool, maintaining academic integrity has become a significant challenge for educators. This paper presents strategies to tackle this issue, focusing on the RoBERTa-CNN model for continuous value predictions, particularly for detecting AI-generated text. The model utilizes the RoBERTa transformer architecture to generate contextualized token embeddings, which are aggregated using mean pooling to create sequence-level representations. The study provides an in-depth mathematical framework for the hybrid RoBERTa-CNN model, covering key components such as input tokenization, self-attention mechanisms, pooling strategies, and classification transformations. The performance outperforms state-of-the-art results, with the RoBERTa-CNN model showing a 1–25% improvement over BERT. The model achieves 100% overall accuracy in identifying human-written text, significantly reducing false positives and ensuring that genuine human content is not misclassified as AI-generated. Evaluation metrics validate its effectiveness in minimizing false negatives, including Recall, F2 score, perplexity, human evaluation, BLEU, ROUGE, and diversity scores. The proposed hybrid transformer-based language model and Machine Learning (ML) classifier enable high-accuracy detection of AI-generated text, ensuring that human-written text are correctly identified. Comparative analysis with trending AI text detection tools, such as DetectGPT, GPTZero, Copyleaks, Fast-DetectGPT, and OpenAI's models, demonstrates superior performance. A novel n-gram bag-of-words (BOW) discrepancy language model is introduced, providing educators with valuable tools to uphold academic integrity in the age of advanced AI technologies.

Keywords AI-generated text · Classification tasks · Transforms · Large language models · Deep learning · NLP

1 Introduction

The rapid advancements in artificial intelligence (AI) [1], particularly with generative models like OpenAI's ChatGPT [2], have brought profound changes to numerous sectors, from technology and healthcare to entertainment and education [3]. These AI systems, powered by large language models (LLMs), exhibit remarkable capabilities in understanding and generating human-like text, which has led to

their widespread adoption for various applications, such as content creation, customer service, and language translation. While using these tools offers several benefits, especially in automating complex tasks, it has also raised significant concerns about their implications for academic integrity. One of the most pressing issues in education today is the potential misuse of AI tools to generate text, assignments, and other forms of academic work, thus undermining the core principles of education, including fairness, originality, and intellectual honesty. Generative AI tools, particularly those that utilize LLMs, are trained on vast datasets derived from diverse sources such as books, websites, and articles, enabling them to produce coherent, contextually appropriate, and seemingly original content [4]. These AI models have demonstrated an impressive ability to produce high-quality text that mimics human writing, making it increasingly difficult to distinguish between AI-generated and human-authored content. This challenge has created a need for advanced detection methods capable of identifying

✉ Santos Kumar Baliarsingh
santos.baliarsingh@kiit.ac.in

Manish Prajapati
2381040@kiit.ac.in

Prabhu Prasad Dev
prabhu.dev@kiit.ac.in

¹ School of Computer Engineering, KIIT Deemed to be University, Patia, Bhubaneswar 751024, Odisha, India

the subtle linguistic patterns and structural cues that indicate whether a piece of text has been written by an AI model or a human. In academic settings, where students are expected to demonstrate their knowledge and critical thinking skills through writing, the proliferation of AI-generated text poses significant ethical, educational, and practical challenges [5].

Traditional methods of plagiarism detection, such as those employed by tools like Turnitin, rely on identifying direct matches between submitted text and previously published sources. While these tools can effectively detect instances of literal copying, they are ill-equipped to identify more sophisticated forms of academic dishonesty, such as the use of AI tools to generate original but non-human content. AI-generated text do not typically match existing sources in the same way that traditional plagiarism might, and thus, they often escape detection by conventional plagiarism-detection systems. This has led to a growing recognition that new, more advanced techniques are needed to identify AI-generated text, particularly in the context of academic submissions. In response to these challenges, this paper proposes a novel approach to AI text detection using a RoBERTa-CNN model. RoBERTa [6], which stands for Robustly optimized BERT (Bidirectional Encoder Representations from Transformers), is a state-of-the-art language model that has been shown to outperform many other pre-trained models in a variety of natural language processing (NLP) tasks, such as text classification, sentiment analysis, and question answering. Unlike traditional models, RoBERTa excels in capturing the contextual relationships between words and phrases, which makes it highly effective for understanding the nuances of language and, consequently, distinguishing between human- and machine-generated content.

The core idea behind the proposed RoBERTa-CNN model is to leverage the power of RoBERTa's contextualized token embeddings to predict continuous values associated with text, such as the likelihood that a given piece of text was generated by an AI model. This classification-based approach differs from traditional classification models that simply classify text as either "human" or "AI." Instead, the model outputs a scalar value that can represent the degree of confidence in the AI authorship of the text. The model achieves this by processing the text through RoBERTa's self-attention mechanism [7], which captures intricate dependencies between words and generates meaningful embeddings. These embeddings are then aggregated using a mean pooling strategy to form a fixed-size representation of the entire text, which is passed through a multi-layer classification head to produce the final output. The adoption of classification-based modeling is particularly beneficial in AI text detection, as it allows for a more nuanced and probabilistic approach to distinguishing between AI-generated and human-written content. This can help avoid the pitfalls

of binary classification, where text that are borderline or exhibit characteristics of both human and machine authorship may be misclassified. The continuous scoring system provided by the classification model allows for a more refined assessment of the likelihood of AI involvement, providing educators with a more reliable tool for detecting AI-generated text. Unlike traditional classification tasks, where the output is typically a discrete class label ("human" or "AI"), classification tasks predict continuous values. In this context, we aim to develop a system that can score or rank the likelihood that a piece of text is AI-generated, providing educators with a reliable method for detecting academic misconduct. To achieve this, the study combines the representational strength of RoBERTa with a classification head that maps these rich embeddings to scalar outputs. Additionally, the model incorporates regularization techniques, such as dropout and batch normalization, to enhance generalization and reduce overfitting during training.

These models and tools are commonly employed in the detection of AI-generated text. Logistic regression (LR) [8] and support vector machine (SVM) [9] are traditional ML models used for classification tasks, while the naive Bayes classifier (NBC) [10] offers a probabilistic approach. The combination of LR with XGBoost enhances predictive performance by leveraging gradient boosting alongside logistic classification. Recurrent Neural Networks (RNN) [10], Bidirectional Gated Recurrent Units with Attention (BiGRU-Attention) [11], and Long Short-Term Memory (LSTM) networks with Attention and CNN layers are advanced deep learning models designed for sequential data processing and enhanced feature extraction [12]. Hybrid models like Mamba-SSM-Attention combine self-supervised models (SSM) with attention mechanisms to boost accuracy [13]. Transformer-based models such as XLNet [14] and RoBERTa [7], when coupled with CNN layers [15], further elevate performance by handling complex dependencies in the data. Tools like OpenAI-Detector [16], GPTZero [17], DetectGPT [18], Copyleaks [19], and Fast-DetectGPT [20] provide practical solutions for detecting AI-generated text by leveraging ML and NLP techniques.

Research Question:

The primary research question of this study is: How can the hybrid RoBERTa-CNN model be effectively utilized for detecting AI-generated text, such as text written by ChatGPT, and how does it compare to existing AI text detection tools in terms of accuracy and reliability?

Here are five key contributions to the paper.

- **Quantification of RoBERTa and CNN Contributions**—Conducts an ablation study to measure the individual impact of RoBERTa and CNN on overall model performance.

- **RoBERTa-CNN Model for AI text Detection:** Introduces the use of the RoBERTa-CNN model for detecting contributions, leveraging the power of transformer architecture for improved accuracy.
- **Mathematical Framework:** Provides a detailed explanation of the hybrid RoBERTa-CNN model's components, including tokenization, attention mechanisms, and classification layers.
- **100% accuracy in identifying human-written text:** The performance outperforms state-of-the-art results, with the RoBERTa-CNN model showing a 1-25% improvement over BERT. Achieves perfect accuracy in distinguishing human-generated text, minimizing false positives.
- **Comparison with Existing Tools:** Comparative analysis with trending AI text detection tools, such as DetectGPT, GPTZero, Copyleaks, Fast-DetectGPT, and OpenAI's models, demonstrates superior performance.
- **Novel N-gram BOW Discrepancy Model:** Introduces a new n-gram BOW discrepancy language model for enhanced AI-human text distinction.
- **Practical Implications for Academic Integrity:** Provides actionable tools and insights to help educators detect AI-generated text and maintain academic integrity.

Following the preliminary section, the paper is structured as follows: Sect. 2 provides the literature review, while Sect. 3 outlines the background. Section 4 describes the datasets, and Sect. 5 explains the methodology. The results and discussions are presented in Sect. 6. Lastly, Sects. 7 and 8 cover the study's limitations, conclusion, and future research prospects.

2 Related work

Before initiating this project, our team conducted a thorough review of several influential research papers to gain a deeper understanding of existing methodologies for AI-generated text detection. Among the numerous studies analyzed, two papers proved to be particularly instrumental in shaping our approach: "GTLR: Statistical Detection and Visualization of Generated text" [3] and "Can AI-generated text be Reliably Detected?" [21]. These works provided essential insights that informed the development of our detection framework, particularly in terms of statistical analysis, interpretability, and the practical implementation of AI text detection techniques.

Detecting AI-generated text is a crucial task in natural language processing (NLP) and plays a significant role in ensuring the authenticity of written content in various domains, including education, journalism, and content

moderation. The proliferation of advanced generative language models has heightened the need for robust detection mechanisms to differentiate between human-authored and AI-generated text. In this section, we explore the Generative textual Likelihood Ratio (GTLR) approach, a method specifically designed to enhance AI text detection by leveraging statistical analysis and visual representation [3].

The GTLR approach operates by computing likelihood ratios that quantify the probability of a given text being generated by an AI model. This probability estimation is a key component in assessing whether a text aligns more closely with AI-generated patterns or human-written structures. To enhance the interpretability of these results, the GTLR framework introduces the Log Likelihood Ratio Plot (LLRP), a visual tool that graphically represents likelihood ratios. The LLRP aids in making complex statistical findings more accessible and comprehensible, allowing researchers and practitioners to analyze AI-generated text with greater clarity.

At the core of the GTLR methodology are two distinct models that work together to evaluate text authenticity:

1. **Generative Model:** This model is trained on AI-generated text and is designed to capture the linguistic characteristics and patterns inherent in machine-produced text. By analyzing such characteristics, the model estimates the likelihood that a given text was generated by an AI system.
2. **Discriminative Model:** Unlike the generative model, the discriminative model is trained on human-written text. Its primary function is to distinguish between human- and AI-generated writing by identifying linguistic features that are more commonly associated with human authorship.

By employing both the generative and discriminative models, the GTLR approach establishes a comprehensive framework for AI text detection. The likelihood ratios computed from these models serve as a statistical foundation for assessing whether a piece of text is AI-generated. The integration of the LLRP visualization further strengthens the framework by providing a clear and interpretable representation of probability distributions. This dual approach ensures a more reliable and transparent detection system, facilitating the identification of AI-generated content with greater accuracy.

The GTLR framework is particularly valuable due to its adaptability across different AI text generation models. As large language models continue to evolve, incorporating more sophisticated generation techniques, detection methodologies must also advance accordingly. The statistical basis of GTLR allows it to remain relevant and effective, even

as AI-generated text becomes more nuanced and human-like. Additionally, the use of visual analytics through LLRP enhances the usability of the system, making it a practical tool for both researchers and educators seeking to verify the authenticity of textual content.

2.1 The advancement of language models

The development of language models has played a transformative role in natural language processing (NLP), enabling machines to extract meaningful insights from vast amounts of unstructured text. Over the years, NLP has evolved through several key milestones, fundamentally altering how computers comprehend and process human language. Early NLP models primarily relied on probabilistic methods, which involved learning word occurrence probabilities from extensive text corpora [22]. However, these early techniques were often limited in capturing deeper semantic relationships between words.

A breakthrough came in 2013 with the introduction of Word2Vec, which revolutionized NLP by converting words into dense vector representations known as word embeddings [23]. This innovation allowed models to identify semantically similar words by analyzing their contextual usage within large text corpora. Despite its effectiveness, Word2Vec struggled with polysemy—the phenomenon where a single word has multiple meanings—and could not capture contextual nuances [24]. This limitation spurred further research into more advanced embedding techniques.

To overcome these challenges, researchers explored pretraining strategies and context-aware language models, which led to the development of dynamic word representations. Unlike static embeddings, these models leveraged contextual information to generate word representations that changed based on their surrounding text [25, 26]. This shift significantly improved language understanding, paving the way for more sophisticated NLP applications. A pivotal advancement in this area was the creation of ELMo, which utilized a bidirectional Long Short-Term Memory (biLSTM) network to generate context-dependent word embeddings, thereby enhancing the ability to distinguish word meanings in varying context [25].

The introduction of the Transformer architecture in 2017 marked another major leap forward [27]. Unlike recurrent models such as LSTMs, the Transformer architecture employed a self-attention mechanism, allowing it to capture long-range dependencies more effectively. This breakthrough facilitated the development of more powerful NLP models, with Bidirectional Encoder Representations from Transformers (BERT) emerging as a prominent example in 2018 [28]. By processing text bidirectionally, BERT significantly enhanced contextual understanding and semantic

interpretation, leading to improved performance across various NLP tasks. Subsequent models, such as RoBERTa [6] and T5 (text-to-text Transfer Transformer) [29], further refined Transformer-based methodologies, expanding their applicability to text classification, translation, and summarization.

The landscape of NLP was further transformed by the advent of the Generative Pre-trained Transformer (GPT) models, introduced by OpenAI [30]. These models, which focused primarily on the decoder component of the Transformer architecture, set new benchmarks in language generation and popularized the “pretraining and fine-tuning” paradigm. This framework inspired a new generation of models, including BART [31], RoBERTa [6], and T5 [29], which optimized pretraining techniques to enhance performance across tasks such as question answering, sentiment analysis, and named entity recognition.

Recent advancements in computational power and the availability of large-scale datasets have given rise to large language models (LLMs), marking a significant departure from traditional NLP methodologies [32]. Notably, GPT-3 [33] and GPT-4 [34] have demonstrated remarkable linguistic capabilities, redefining human-AI interaction through sophisticated natural language generation. OpenAI’s ChatGPT, built upon GPT-3 and GPT-4, has established itself as a leading conversational AI system, exhibiting an unprecedented level of fluency and coherence. Meanwhile, alternative models such as LLaMA [35] have been developed with a focus on efficiency and reduced computational requirements, making them valuable for various text-generation applications.

As NLP continues to evolve, ongoing research aims to enhance model interpretability, reduce biases, and optimize computational efficiency. The integration of hybrid architectures combining Transformers with novel neural architectures presents promising avenues for future advancements in language modeling.

2.2 Evolution of prompting strategies in ChatGPT

Traditional NLP models have heavily relied on supervised learning, which necessitates large labeled datasets for effective training. However, the advent of prompt engineering, also known as prompt learning, has significantly transformed this paradigm by reducing the dependence on extensive fine-tuning. Instead of requiring direct model updates through gradient-based learning, this approach harnesses the power of large language models (LLMs) like ChatGPT by designing well-structured prompts that guide the model’s responses. By framing queries effectively, researchers can elicit high-quality outputs in zero-shot or few-shot settings,

enabling ChatGPT to perform diverse NLP tasks with minimal training data [36], [37], [38].

ChatGPT has gained widespread attention across multiple disciplines due to its ability to generate human-like text with remarkable fluency. It has been particularly useful in fields such as healthcare, scientific research, and education, where it assists in academic writing [39], [40], research topic generation [41], [42], and domain-specific text translation [43]. Its accessible interface and adaptability have made it an invaluable tool for professionals seeking to enhance productivity and streamline content generation.

Despite its advantages, ChatGPT is not without its challenges. Issues such as hallucinations, plagiarism, and misinformation have raised concerns about its reliability in academic and professional settings. The generation of fabricated citations and misleading data has been particularly problematic, emphasizing the need for human oversight and verification of its outputs. Additionally, ethical considerations surrounding AI-generated content call for stricter regulation and responsible AI usage to prevent misinformation and academic dishonesty [44], [45].

As prompt engineering continues to evolve, refining strategies for effective model interaction remains a key area of research. Future developments aim to enhance the precision, reliability, and contextual understanding of AI-generated responses, ensuring that ChatGPT and similar models can be leveraged responsibly and effectively across various domains.

2.3 The progression of the GPT models

The Generative Pre-trained Transformer (GPT) series, developed by OpenAI, has significantly transformed the field of natural language processing (NLP) through its advanced approach to language modeling. The introduction of GPT-1 marked a pivotal shift from traditional NLP techniques by utilizing unsupervised learning on the BookCorpus dataset to understand long-range linguistic dependencies. This breakthrough laid the foundation for fine-tuning on downstream NLP tasks, allowing the model to generate coherent and contextually relevant text. Unlike earlier Long Short-Term Memory (LSTM) networks, GPT-1 employed a 12-layer decoder-only Transformer, leveraging self-attention mechanisms to improve text generation capabilities.

The next milestone in this evolution came with GPT-2, introduced in 2019, featuring 1.5 billion parameters and a significantly larger training corpus drawn from 8 million documents (approximately 40 GB) sourced from Reddit [46]. GPT-2 showcased exceptional performance in generating text without requiring labeled training data, demonstrating the power of zero-shot learning. By predicting the

next word in a sequence based on vast amounts of diverse, unsupervised data, GPT-2 surpassed baseline NLP systems, proving that massive-scale pretraining could enhance language understanding without task-specific fine-tuning. However, despite its remarkable fluency, GPT-2 exhibited limitations, including the generation of incoherent, biased, or factually incorrect outputs [33].

To overcome these challenges, OpenAI introduced GPT-3, an exponentially larger model boasting 175 billion parameters and trained on 400 billion byte-pair-encoded tokens. The increase in scale enabled GPT-3 to perform various NLP tasks, such as question answering, text completion, and common-sense reasoning, with minimal or no training data. Its ability to understand and generate human-like text in zero-shot and few-shot scenarios sets a new benchmark for language models, cementing its role as a fundamental tool in AI-driven applications. Despite its capabilities, GPT-3 faced persistent issues, including bias, fabricated content, and difficulty adhering to user instructions [47], [48].

Recognizing the need for better alignment with human intent, OpenAI introduced InstructGPT, a fine-tuned variant of GPT-3 that incorporated reinforcement learning from human feedback (RLHF). With a reduced model size of 1.3 billion parameters, InstructGPT was trained on carefully curated datasets containing human-annotated demonstrations, improving its ability to follow user instructions accurately. The training process included policy optimization and reward modeling, allowing the model to generate more reliable, context-aware, and ethical responses while minimizing harmful biases [49].

Building upon the advancements of GPT-3 and InstructGPT, OpenAI developed ChatGPT, a conversational AI model designed for interactive and dynamic dialogues. This innovation enhanced AI-generated responses by ensuring they aligned with user expectations across various domains, including healthcare [50], scientific research, and engineering design. ChatGPT's ability to understand context, engage in multi-turn conversations, and provide insightful responses revolutionized human-AI interactions, making it a widely adopted tool for both personal and professional applications.

The latest breakthrough in the GPT series, GPT-4, released in 2023, further refined language understanding and problem-solving capabilities. Retaining RLHF-based alignment, GPT-4 exhibited improved factual accuracy, reduced biases, and enhanced contextual awareness, making it more reliable than its predecessors [34]. Unlike earlier models that required precise prompting, GPT-4 seamlessly handled complex queries while maintaining coherence and logical consistency, positioning it as a major step toward Artificial General Intelligence (AGI).

GPT-4's integration into ChatGPT Plus has significantly enhanced the conversational AI experience, offering users more refined, context-aware interactions. With its scalability, efficiency, and improved safety measures, GPT-4 is reshaping the future of AI-driven communication, setting new standards for natural language understanding and human-computer interaction.

3 Background

3.1 Transformer architecture

As stated by Vaswani [27], a Transformer-based language model is made up of several layered Transformer blocks. Each block consists of a fully linked positional feed-forward network after a multi-head self-attention layer. The conventional self-attention mechanism's inability to automatically recognize the positioning information of words inside a sequence is one of its drawbacks. Positional bias is applied to each input word embedding to address this, enabling each word to be represented by a vector that takes into account both its location in the sequence and its content. Usually, either absolute position embedding ([28, 30]) or relative position embedding ([51, 52]) is used to implement this positional bias. Studies have indicated that activities about natural language creation and comprehension are better served by relative position representations [52].

In contrast to other techniques, the suggested disentangled attention mechanism presents a fresh approach. Each input word in this mechanism is represented by two

different vectors, one of which encodes the word's location and the other its content. Then, using disentangled matrices that handle the content and relative position information independently, attention weights between words are determined. A more sophisticated method of capturing the link between words in a sequence while maintaining their places is offered by this disentangled technique.

3.2 Masked language model

The Masked Language Model (MLM) [53] is a self-supervision objective commonly used to train large-scale pre-trained Transformer-based models (PLMs) on massive volumes of text data. According to this method, a random selection of 15% of the tokens in a sequence of tokens $X = (x_1, x_2, \dots, x_n)$ are masked, and the model is trained to predict the missing tokens. Using the context that the unmasked tokens give, the formal goal is to maximize the log-likelihood of the masked tokens (see equation 1):

$$\max \log p(X|X_{\text{mask}}) \quad (1)$$

Where X is the collection of masked token indices in the sequence? 10% of the masked tokens in the original BERT model stay the same, 10% are swapped out for randomly selected tokens, and the remaining 80% are swapped out for the unique [MASK] token. Through training on a huge corpus of text without the need for labeled data, this method allows the model to acquire contextualized word representations, allowing it to capture rich semantic information across a variety of language tasks.

Table 1 Category-wise Distribution of Questions, Human Answers, and AI Answers

Category (Sources)	# of Questions	# of Student Answers (0)	# of AI Answers (1)
Persuade Corpus	103,984	103,984	98,345
ChatGPT	9,684	18,654	17,337
LLaMA2-Chat	9,684	19,587	7,561
Mistral 7B v2	4,842	14,842	6,660
Mistral 7B v1	4,842	4,842	5,842
Original-Moth	7,263	6,842	6,842
Train-text [54]	4,134	14,134	3,842
LLaMA 70B v1	3,516	3,842	3,842
Falcon 180B v1	3,165	3,842	2,842
Claude-v7	1,000	1,000	1,000
Claude-v6	1,000	942	942
GPT-3.5	1,500	1,500	1,342
LLaMA-2-7B-Chat	842	842	842
Cohere-Command	1,000	942	1,742
PaLM-text-Bison1	1,842	1,842	2,845
GPT-4	4,842	14,842	6,842
Total	161,640	212,479	169,668

4 Datasets

The experiments conducted in this study utilize a public dataset, the "EnglishQA text Corpus," comprising 161,640 questions, each paired with responses generated by both human experts and various AI models, including ChatGPT, Llama2, Mistral7 b, and others (see figure 3). The dataset spans a diverse range of question categories, with a particular emphasis on English and Hindi responses for analytical purposes. Table 1 provides a detailed breakdown of the dataset's distribution across different subcategories. Figure 1 illustrates the distribution of AI-generated and student-written responses across various categories. Student responses comprise 55.7% of the dataset, while AI-generated answers constitute 44.3%. The AI segment is slightly separated to emphasize its proportion. This visualization helps analyze the prevalence and impact of AI-generated content.

ChatGPT was prompted in a fresh, blank conversation for each question to ensure the integrity of AI-generated

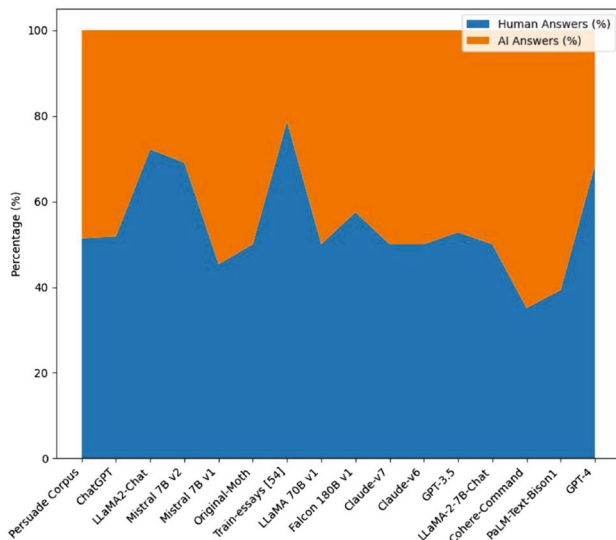


Fig. 1 Category-wise Distribution of Human vs. AI-Generated Answers (%)

responses, preventing any influence from prior interactions. The dataset was split into training (80%) and testing (20%) sets, with a primary focus on experimental analysis rather than data collection. This structured approach facilitates a clear comparative evaluation of AI-generated and human-written text, offering valuable insights into the effectiveness of AI-generated text detection. Figure 2 presents a bar chart

illustrating the distribution of dataset entries across various sources, comparing the number of questions, human-written answers, and AI-generated answers. The largest portion comes from the Persuade Corpus, with over 100,000 entries in each category. Other significant sources include AI models such as ChatGPT, LLaMA2-Chat, and GPT-4. The chart underscores the diversity of the dataset, encompassing content generated by both students and different large language models (LLMs). This diversity is crucial for ensuring robust training and evaluation in tasks related to AI-generated text detection.

4.1 Sample case

In an experiment, teachers were unable to distinguish between AI-generated text and those written by humans. In this study, we present two excerpts (Table 2) to highlight the capability of ChatGPT in generating text that closely resemble those written by students. One excerpt was written by the author during their time as a postgraduate student at KIIT University, while the other was produced by ChatGPT. Both paragraphs address the same text's question about data-driven predictive policing. Can you tell which excerpt was written by the human student? More significantly, if a student submits text that you believe were created by AI, what actions can you take to prove your suspicion? This

Fig. 2 Distribution of Dataset Sources—Student vs. AI Responses

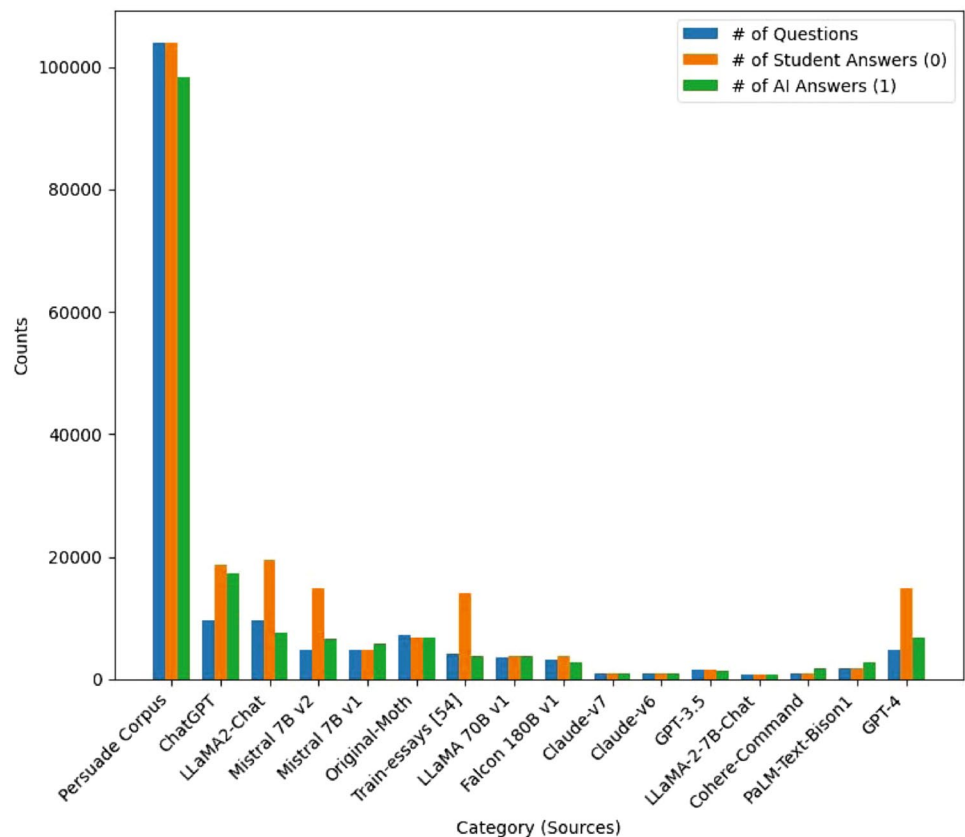


Table 2 Comparison of Former Student's and ChatGPT's Answers on NLP

Question	Student's excerpt (Answer)	ChatGPT's excerpt (Answer)
What is NLP, and how do I write applications?	NLP is a subfield of artificial intelligence that focuses on enabling computers to interpret and process human language. It is used in many areas, such as search engines and voice recognition systems, to help computers understand text or speech. NLP uses techniques like ML, statistical models, and deep learning to perform tasks like sentiment analysis, translation, and text summarization. One of the challenges in NLP is the ambiguity of natural language—words can have multiple meanings depending on the context. Overcoming this challenge is key to building more effective NLP systems	NLP is a specialized area of AI that deals with the interaction between computers and human language. NLP involves enabling machines to process, understand, and generate human language in a way that is both useful and meaningful. This includes a wide range of tasks, such as speech recognition, text classification, machine translation, and chatbot interaction. To achieve this, NLP combines computational linguistics with ML models and deep learning techniques. By doing so, it enables machines to parse complex sentences, analyze sentiment, and even translate languages with a high degree of accuracy. However, NLP faces challenges like language ambiguity and contextual meaning, which require advanced algorithms to address

N = 161,640 (212,479 human-generated; 169,668 AI-generated),
Source: Authors' own creation

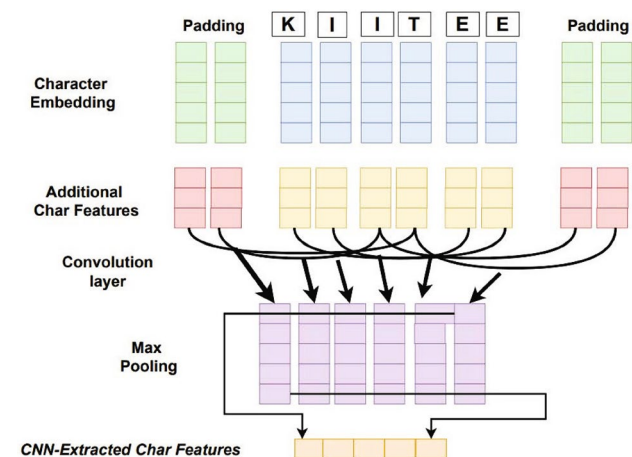


Fig. 3 The CNN extracts character-level features from each word. Character embeddings and optional character-type feature vectors are generated using lookup tables, concatenated, and then passed through the network

subject raises serious concerns regarding academic integrity as AI technologies grow more competent at imitating human writing styles. Detecting AI-generated text necessitates a rigorous examination of writing patterns, consistency, and potential external validation methods.

4.2 Data augmentation

The EnglishQA Essays Corpus comprises text from middle school, high school, undergraduate, and postgraduate students, along with AI-generated responses. To enhance dataset quality, balance, and robustness, we applied diverse data augmentation techniques that reflect variations in writing styles across academic levels. For human-written text, we used strategies such as synonym replacement, substituting words with contextually appropriate synonyms to increase lexical diversity, and back-translation, which introduces sentence structure variations through language translation cycles. Paraphrasing techniques using NLP models generated alternative sentences while preserving meaning, and sentence expansion or compression helped simulate writing complexity across student groups.

To augment AI-generated text, we employed prompt engineering, modifying inputs to guide AI outputs toward different academic levels. We also adjusted generation parameters (temperature, top-k, top-p) to control fluency and coherence. Fine-tuned generation aligned AI responses with specific writing styles, while human-in-the-loop editing added natural linguistic variation, making outputs more human-like.

These augmentation strategies helped balance the dataset, reduce bias, and improve generalization. The enriched corpus captures realistic writing diversity, enhancing the performance of AI-generated text detection models. Ultimately, this supports research in academic integrity, responsible AI use, and the development of fair and accurate AI detection systems for educational settings.

4.3 Text preprocessing

Text preprocessing is a very important stage of NLP; it helps clean and prepare the text data further for analysis or modeling. This work is successively done with the help of converting to lowercase, word splitting, stop-word removal, and superfluous spaces [55]. The language in this document is cleaned up first, which means extraneous letters, punctuation, and other distracting elements are removed to make it cleaner. Second, the entire text is converted into lowercase, thereby ensuring that words that are otherwise the same are not treated as different just because of their case. Following this is the step of tokenizing the text, where it is broken down into individual words or tokens, forming the basis

for further processing of text data. The input is a sequence of tokens, with special tokens [CLS] at the beginning and [SEP] at the end to indicate sentence boundaries. [CLS] is a classification token, and its hidden state will be used as the sentence representation after the model processes the entire sequence. Token Embeddings: Each token ("cars", "have," etc.) is converted into a token embedding using the WordPiece embedding technique. The tokens [CLS] and [SEP] are also embedded in the same way. Segment embeddings distinguish between different sentences in the input. In this example, segment A is used for the first sentence (green blocks), and segment B is used for the second sentence (blue blocks). These embeddings encode the position of each token in the sentence. Since transformers do not have inherent knowledge of token positions, positional embeddings (E_0, E_1 , etc.) are added to give the model positional awareness. Position embeddings are randomly initialized and learned over time. The final input representation is obtained by adding together the token embeddings, segment embeddings, and positional embeddings for each token. The [CLS] token's embedding (highlighted) represents the entire sentence after processing and can be used for classification or detection tasks.

5 Methodology

This section explains the pre-training methodology of RoBERTa and the steps applied to fine-tune this language model for our classification task.

Introduced by Facebook AI in 2019, the RoBERTa model represents a significant advancement over its predecessor, BERT [28], through a series of key modifications. While both models share the same Transformer architecture and pre-training/fine-tuning paradigm, RoBERTa outperforms BERT due to its larger training corpus, expanded BPE (Byte Pair Encoding) vocabulary, the removal of the Next Sentence Prediction (NSP) task, the use of dynamic masking, and training with large mini-batches.

Compared to BERT, RoBERTa was pre-trained on a much larger corpus, incorporating BOOKCORPUS plus English WIKIPEDIA (16GB), CC-NEWS (76GB), OPEN-WEBtext (38GB), and STORIES (31GB) [6]. The extended pre-training on a more diverse and comprehensive corpus allows RoBERTa to capture deeper insights into human language patterns.

One of the primary improvements in RoBERTa is the removal of the NSP task from the training objectives, which helps boost performance on downstream tasks. A detailed explanation of how this modification improves RoBERTa's end-task performance can be found in the model architecture section. Another key enhancement is the implementation of

dynamic masking, which differs from BERT's static masking approach. In static masking, a fixed set of tokens is randomly masked during pre-training, whereas in dynamic masking, a different set of tokens is randomly masked in each iteration of pre-training. This approach prevents the model from overfitting to specific patterns, improving generalization.

RoBERTa also benefits from training with large mini-batches. The creators of RoBERTa found that training with large batches significantly improves perplexity and end-task accuracy, as the model can better learn from the data during each epoch. Mini-batch training, a common deep learning technique, updates the model's weights multiple times per epoch, leading to faster convergence and improved performance.

Fine-tuning the RoBERTa model begins with converting the EnglishQAtext Corpus dataset (in JSONL format) into a Pandas dataframe. This dataframe contains four columns: "Question," "text," "LabelName," and "Label." The "Question" column holds the question prompts, while the "text" and "LabelName" columns store the input text and their corresponding labels (Human Answer or ChatGPT Answer). The "Label" column contains binary labels (0 or 1), where 0 represents human-written text and 1 represents AI-generated text. Next, the dataset is split into training and test sets in an 80% and 20% ratio, respectively.

The input text data is then tokenized using the RoBERTa tokenizer from Hugging Face, and the model used is RoBERTa-base. Tokenized sequences are assigned a maximum length of 128 and are either truncated or padded to meet the input size requirements of RoBERTa. The tokens are then converted into tensors. Like BERT, RoBERTa has 12 layers of transformer blocks with a hidden size of 768. For optimization, we use the Adam optimizer and cross-entropy loss. During the training process, we fine-tuned hyperparameters, including the learning rate, batch size, and the number of training epochs, to achieve the best performance. After several iterations, the optimal settings were found to be a batch size of 6, a learning rate of $1e-6$, and training for 5 epochs on the EnglishQAtext Corpus dataset.

5.1 The foundation of RoBERTa

Sure! Here's a corrected and polished version of your paragraph with improved clarity and flow, plus a bit more precise math notation for the input sequence and its projection:

RoBERTa inherits its architecture from BERT, which is based on the Transformer model introduced by Vaswani et al. [27]. The Transformer relies on self-attention mechanisms and feed-forward networks to compute contextualized representations of input sequences. RoBERTa uses only the encoder portion of the Transformer, which consists

of L identical layers. Each layer contains two main components: a Multi-Head Self-Attention Mechanism and a Position-Wise Feed-Forward Network.

RoBERTa builds upon BERT's Transformer encoder by leveraging a series of mathematically grounded components to generate deep contextualized representations. At the core of this model is the multi-head self-attention mechanism, which enables each token in the input sequence to attend to every other token, capturing rich contextual dependencies.

Given an input sequence of tokens $X = [x_1, x_2, \dots, x_n]$, where each $x_i \in \mathbb{R}^d$ is a token embedding, the model projects X into three distinct spaces—queries Q , keys K , and values V —using learnable weight matrices, W^Q , W^K , and W^V :

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \quad (2)$$

These matrices—queries (Q), keys (K), and values (V)—are essential for computing how much attention each token should pay to the others. The scaled dot-product attention then computes attention scores (see equation 3):

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V \quad (3)$$

Here, $\sqrt{d_k}$ is a scaling factor to prevent large dot products from destabilizing training. RoBERTa uses multi-head attention, allowing the model to learn attention distributions from multiple representation subspaces (see equation 4):

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O \quad (4)$$

Each head operates in parallel, and the outputs are concatenated and projected through another weight matrix W_O . Following attention, each token is passed through a feed-forward network (FFN) applied independently (see equation 5):

$$\text{FFN}(z) = \text{ReLU}(zW_a + b_a)W_b + b_b \quad (5)$$

To stabilize and accelerate learning, RoBERTa incorporates residual connections and layer normalization around both the attention and FFN layers (see equation 6):

$$z_{\text{out}} = \text{LayerNorm}(z_{\text{in}} + \text{SubLayer}(z_{\text{in}})) \quad (6)$$

The input representation to RoBERTa combines token, positional, and segment embeddings (see equation 10):

$$E(x_a) = E_{\text{token}}(x_a) + E_{\text{pos}}(a) + E_{\text{segment}}(s) \quad (7)$$

RoBERTa is pretrained using a Masked Language Modeling (MLM) objective. 15% of the input tokens are masked, and the model learns to predict them (see equation 8):

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in C} \log P(x_i | X_{\setminus i}) \quad (8)$$

This encourages the model to understand context bidirectionally. For optimization, RoBERTa uses Adam with weight decay, which combines momentum and regularization (see equation 9):

$$\theta_{t+1} = \theta_t - \eta \cdot \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda \cdot \theta_t \right) \quad (9)$$

Here, $\hat{m}_t$ and $\hat{v}_t$ are the bias-corrected first and second moment estimates, η is the learning rate, and λ is the weight decay coefficient. Together, these equations form the backbone of RoBERTa's robust architecture, enabling it to achieve state-of-the-art performance across diverse NLP tasks.

Fine-tuning RoBERTa involves adding a task-specific layer (classification heads) and training on labeled data. The loss function depends on the task, such as cross-entropy for classification or cross-entropy for classification. RoBERTa's mathematical foundations, combined with its architectural and training optimizations, have established it as a benchmark model for NLP. By refining BERT's pretraining methodology and introducing innovations like dynamic masking and longer training durations, RoBERTa achieves state-of-the-art results across diverse tasks, demonstrating the power of robust, scalable Transformer models.

5.2 Convolutional neural networks (CNNs)

The figure illustrates a CNN architecture for character-level feature extraction, a critical technique in NLP tasks, including AI-generated text detection, text classification, and sentiment analysis. This architecture efficiently captures local syntactic patterns, morphological dependencies, and stylistic artifacts, making it particularly effective for identifying subtle linguistic differences between human-written and AI-generated text. The process begins with character embedding, where each character in the input sequence is transformed into a dense vector representation. This embedding maps each character (K, I, T, E, E) into a fixed-dimensional space, shown as the blue blocks. These embeddings capture the character's identity and contextual information, allowing the CNN to recognize morphological patterns. Padding is applied at both ends of the sequence, ensuring consistent input lengths for CNN processing. Following embedding, additional character features (represented by

red and yellow blocks) are appended to enrich the representation. These additional features may include character types, frequency statistics, or stylistic indicators, helping the model detect patterns such as unusual symbol usage or irregular character spacing, which are common in AI-generated content. Next, the concatenated representation is fed into the convolutional layer, which applies multiple filters (kernels) to the input. Each filter scans over local character patterns, capturing n-gram features (trigrams, quadgrams) that reflect repetitive stylistic artifacts. The mathematical convolution operation is defined as (see equation 10):

$$f_{i,j} = \text{ReLU}(W * c_{i,j} + b) \quad (10)$$

Where:

- $f_{i,j}$ is the output feature map.
- W is the convolution kernel with shape (k, d) , where k is the kernel size.
- $c_{i,j}$ is the character embedding slice beinconvolved?.
- b is the bias term.
- $*$ denotes the convolution operation.
- ReLU (Rectified Linear Unit) is the activation function, applying $\text{ReLU}(x) = \max(0, x)$.

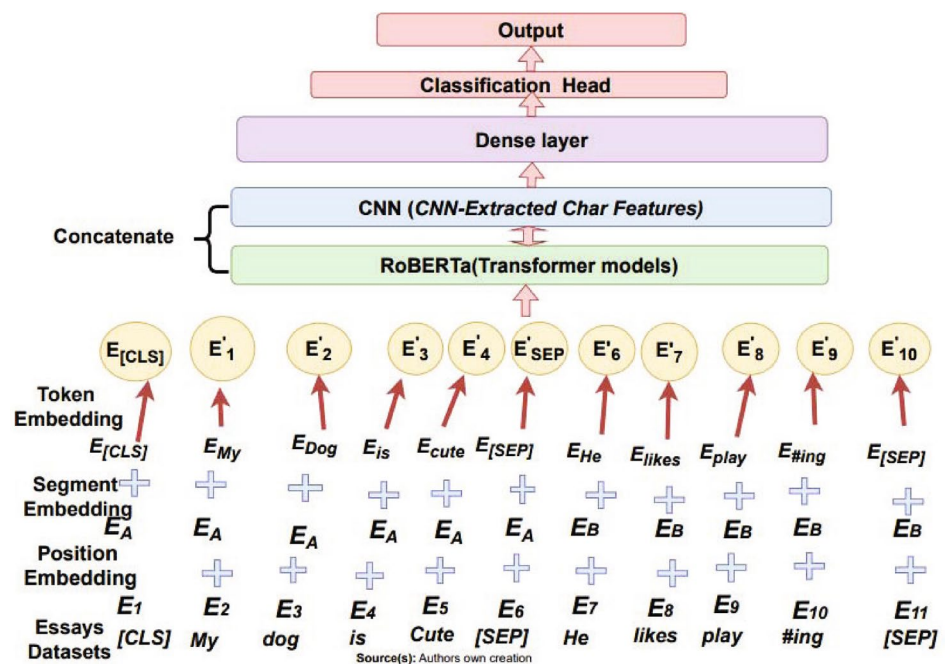
The convolution operation produces multiple feature maps, each detecting different patterns. The model then applies max-pooling, shown as the purple blocks, which reduces the dimensionality of the feature maps by selecting the maximum value from each region. This step enhances computational efficiency and ensures the model focuses on the most significant features. The final output of this pipeline is

the CNN-extracted character feature vector (orange blocks), which contains localized morphological and stylistic patterns. This feature vector is passed to downstream models (BiLSTM, Transformer) for sequence modeling and final classification. The CNN architecture is particularly effective for AI-generated text detection because it captures stylistic artifacts, such as repeated prefixes, unusual punctuation patterns, and character-level anomalies, which are indicative of AI-generated content. By combining character embeddings, convolutional feature extraction, and max-pooling, the model efficiently detects local syntactic patterns and morphological irregularities, making it highly effective for AI-generated text detection and text classification tasks.

5.3 RoBERTa-CNN hybrid language model

The model merges RoBERTa's contextual embeddings with character-level features extracted by CNN. Token, segment, and positional embeddings form the input, processed by RoBERTa and CNN in parallel. Their outputs are concatenated, followed by a dense layer and a classification head for final prediction. Figure 4 illustrates the architecture of a RoBERTa-CNN hybrid model designed for text classification, specifically for distinguishing between human-written and AI-generated text. The model architecture combines character-level features extracted by a CNN with contextual word embeddings generated by the RoBERTa transformer to enhance classification performance. The following sections explain the architecture step by step, including the underlying mathematical intuition and equations.

Fig. 4 RoBERTa-CNN Hybrid Language Model Architecture



5.3.1 Input embedding layer

Given an input sentence:

“My dog is cute. He likes to play #ing”

It is first tokenized into subwords using RoBERTa’s tokenizer:

” text [“<s>”, “My”, “dog”, “is”, “cute”, “</s>”, “He”, “likes”, “play”, “#”, “ing”, “</s>”] ““

Each token is converted into a token embedding vector. transformer-based models like RoBERTa, the final input embedding for each token is a sum of three embeddings:

Equation 11:

$$E_i = E_{\text{token}}(x_i) + E_{\text{segment}}(x_i) + E_{\text{position}}(i) \quad (11)$$

- $E_{\text{token}}(x_i)$: WordPiece token embedding
- $E_{\text{segment}}(x_i)$: Segment embedding (e.g., A or B)
- $E_{\text{position}}(i)$: Positional encoding for token i

For example:

$$E_2 = E_{\text{token}}(\text{“My”}) + E_A + E_2 \quad (12)$$

The input to RoBERTa is thus a matrix:

$$X = [E_1, E_2, \dots, E_{11}] \in \mathbb{R}^{n \times d} \quad (13)$$

Where n is the sequence length, and d is the embedding dimension (typically 768 or 1024).

5.3.2 RoBERTa transformer encoder

RoBERTa encodes contextual dependencies through self-attention and feedforward networks. The output of the last transformer layer for each token is:

Equation 14:

$$H_i = \text{TransformerLayer}(E_i) \quad (14)$$

for $i = 1, \dots, n$

The contextual output representations:

$$H = [H_1, H_2, \dots, H_n] \quad (15)$$

This represents the encoded meaning of each word in the context of the full sentence.

5.3.3 CNN on character-level features

Simultaneously, the model extracts character-level features using a CNN. This is useful for capturing morphological patterns like suffixes (‘-ing’, ‘-ed’, etc.), misspellings, or stylistic features – especially useful in detecting LLM-generated text.

Each token x_i is further broken down into characters and passed through a 1D convolutional layer.

Let $x_i^{(char)} \in \mathbb{R}^{l \times d_c}$ be the character embedding matrix for token x_i , where l is the character length and d_c is the character embedding size.

The CNN applies multiple filters over the input:
Equation 16:

$$c_i = \text{CNN}(x_i^{(char)}) \in \mathbb{R}^k \quad (16)$$

where k is the number of filters or output features.

Then, apply max-pooling to obtain a fixed-size character vector per token:

$$C = [c_1, c_2, \dots, c_n] \quad (17)$$

5.3.4 Feature concatenation

The features from RoBERTa and the character-level CNN are concatenated to form a comprehensive token representation:

Equation 18:

$$F_i = [H_i \parallel c_i] \in \mathbb{R}^{d+k} \quad (18)$$

Stacking them yields:

$$F = [F_1, F_2, \dots, F_n] \in \mathbb{R}^{n \times (d+k)} \quad (19)$$

5.3.5 Dense layer and classification head

To aggregate information across all token positions (or just from the ‘[CLS]’ token), we apply a dense layer, followed by a classification head:

Equation 20 (Dense layer):

$$z = \text{ReLU}(W_1 \cdot F_{\text{CLS}} + b_1) \quad (20)$$

Where:

- $W_1 \in \mathbb{R}^{h \times (d+k)}$, $b_1 \in \mathbb{R}^h$
- F_{CLS} is typically the first token’s output

Equation 21 (Classification layer):

$$\hat{y} = \text{Softmax}(W_2 \cdot z + b_2) \quad (21)$$

For binary classification (e.g., human vs AI-generated), softmax reduces to:

$$\hat{y} = \sigma(W_2 \cdot z + b_2) \quad (22)$$

5.3.6 Training objective

The model is trained using binary cross-entropy loss:

Equation 23:

$$\mathcal{L} = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}) \quad (23)$$

Where:

- $y \in \{0, 1\}$ is the ground-truth label (human or AI)
- $\hat{y}$ is the predicted probability

5.4 Summary of workflow

1. Input Sentence $\rightarrow$ Tokenized using RoBERTa tokenizer
2. Embedding Layer: Sum of Token + Segment + Position embeddings
3. RoBERTa Encoder: Generates contextual embeddings H_i
4. CNN Layer: Extracts character-level features c_i
5. Concatenation: $F_i = [H_i || c_i]$
6. Dense + Classifier: Produces label prediction $\hat{y}$
7. Loss Computation: Optimizes parameters using cross-entropy

5.5 Why combine RoBERTa with CNN?

- RoBERTa captures semantic and contextual understanding at the token level.
- CNN captures subword and morphological patterns at the character level.
- Their combination enhances model sensitivity to linguistic clues often present in AI-generated vs. human-written text.

Backpropagation ensures gradients flow through RoBERTa, CNN, and the classification head, allowing the entire model to be fine-tuned.

Require: Input text $X = \{w_1, w_2, \dots, w_n\}$

Ensure: Predicted class label $\hat{y}$

1. **Step 1: Tokenization and Embedding**
2. Tokenize input: $T = \text{Tokenizer}_{\text{RoBERTa}}(X)$
3. Generate input embeddings for each token $t_i \in T$:

$$E_i = E_{\text{token}}(t_i) + E_{\text{segment}}(t_i) + E_{\text{position}}(i)$$

4. **Step 2: Transformer Encoding (RoBERTa)**
5. Pass embeddings through RoBERTa encoder:

$$H_i = \text{RoBERTaEncoder}(E_i)$$

6. Get contextualized embeddings: $H = [H_1, \dots, H_n]$
7. **Step 3: Character-Level Feature Extraction (CNN)**
8. **for** each token t_i in T **do**
9. Convert t_i to character-level embedding matrix: X_i^{char}
10. Apply 1D CNN and max-pooling:

$$c_i = \text{CNN}(X_i^{\text{char}})$$

11. **end for**
12. Obtain CNN features: $C = [c_1, \dots, c_n]$
13. **Step 4: Feature Concatenation**
14. **for** $i = 1$ to n **do**
15. Concatenate: $F_i = [H_i || c_i]$
16. **end for**
17. **Step 5: Classification**
18. Use [CLS] token embedding: F_{CLS}
19. Apply dense layer:

$$z = \text{ReLU}(W_1 \cdot F_{\text{CLS}} + b_1)$$

20. Compute output:

$$\hat{y} = \sigma(W_2 \cdot z + b_2)$$

21. **return** $\hat{y}$

Algorithm 1 RoBERTa-CNN Hybrid Model for text Classification

5.6 Perplexity

In NLP, perplexity is a measure used to evaluate how well a language model predicts a given word based on the context of the preceding words. For instance, in the sentence “The sun rises in the _____,” if the model already knows the words “The sun rises in,” it is highly likely to predict “east” as the next word, but it could also predict “morning” with some probability. Both “east” and “morning” would have relatively high probabilities of occurring next, leading to a low perplexity. In contrast, if the model predicts “piano” after “The sun rises in,” this would result in a higher perplexity because the likelihood of “piano” fitting the context is much lower. Perplexity is computed by multiplying the probabilities of each word given the previous words and then averaging the score by taking the geometric mean. The lower the perplexity, the better the model is at predicting the next word in a sequence.

5.7 Pretraining

Pretraining is the initial phase in building a language model, where a model is trained on a large corpus of text without specific task objectives. This stage helps the model learn general language patterns, structures, and relationships between words across various contexts. For example, a model like RoBERTa can be pre-trained using vast amounts of text data to predict missing words in sentences (masked

language modeling). During pretraining, the model doesn't know what task it will be later used for (text classification or generation). The focus is on enabling the model to learn robust representations of language. Once pre-trained, the model can then be fine-tuned for specific tasks, like detecting AI-generated text, by training it on task-specific datasets. This approach helps to leverage the vast knowledge the model has learned during pretraining, making it more effective when later adapting to specialized tasks.

5.8 Fine-tuning

Fine-tuning refers to the process of adjusting a pre-trained language model to improve its performance on a specific task. After pretraining, the model is adapted by training it further on a smaller, task-specific dataset. The first step is to organize the data into three sets: a training set, a validation set, and a testing set. The model may be augmented with additional layers to refine its capabilities for the specific task, such as adding a classification head for binary classification. Fine-tuning can be done by training only the new layers or by retraining the entire model end-to-end with the training dataset. Hyperparameters such as learning rate and batch size are often adjusted during this phase to improve performance. After fine-tuning, the model is tested using the validation set to ensure it generalizes well, and any necessary adjustments to the model or hyperparameters can be made. Finally, the model is evaluated on the test set to assess its effectiveness in real-world scenarios.

6 Results and discussion

6.1 Experimental setup

We trained our models on a machine equipped with two NVIDIA A100 GPUs. For the base models, using the hyperparameters outlined in the paper, each training step took approximately 0.6 s. We trained the base models for a total of 20,000 steps, which took around 1.5 days. For the larger models (detailed in the bottom row of Table 5), the training step time was 1.0 s. These larger models were trained for 50,000 steps, requiring approximately 3.5 days to complete. This setup allowed us to process and fine-tune our models within the given time frames.

Table 9 presents the hyperparameter configurations for various machine learning and deep learning models used in AI-generated text detection. Traditional models like LR and SVM utilize learning rates of 0.01, with SVM leveraging an RBF kernel and parameters like $C = 1.0$ and $\gamma = \text{'scale'}$. NBC incorporates smoothing of 1.0. Advanced models like CNN, RNN, and BiGRU-Attention use optimizers such as

Adam and RMSprop, with dropout rates ranging from 0.3 to 0.5 to prevent overfitting. Transformer-based models, including BERT, RoBERTa, T5, and their variations, rely on AdamW or Adafactor optimizers, GELU/ReLU activations, and hidden sizes ranging from 512 to 1024. CNN-enhanced Transformer models (BERT-CNN, RoBERTa-CNN, XLNet-CNN) integrate convolutional layers with kernel sizes of 3 and filter sizes between 128 and 256 for feature extraction. The Mamba-SSM-Attention model employs a Swish activation and a state dimension of 256, reflecting its unique sequence state space mechanism. Learning rates vary, with Transformers typically using lower values ($2e - 5$ for BERT), while models like T5 employ higher rates ($1e - 4$). These settings are crucial for optimizing each model's ability to generalize across AI- and human-generated text, ensuring accurate detection performance.

6.2 Direct experiments explanation

In our experiments, we trained 10 variations of a Transformer hybrid model, dividing them into two categories: 5 models with frozen parameters and 5 with unfrozen parameters. Each model was trained with varying portions of the dataset, using 80%, 20%, and 100% of the data, and the results were carefully recorded in a table for comparison. To evaluate the performance of the models, we employed several widely used metrics: precision, recall, and the F2-measure. Precision quantifies how many of the positive predictions made by the model truly belong to the positive class, while Recall measures how many actual positive samples were correctly identified by the model. The F2-measure, which balances precision and recall, provides a more comprehensive evaluation, with values closer to 1 indicating a stronger overall performance. In addition to these metrics, we also assessed accuracy, perplexity, and binary cross-entropy to evaluate the model's ability to classify AI-generated text accurately and to measure the model's prediction consistency. Perplexity serves as an indicator of how well the model predicts the next word in a sequence, with lower values indicating better performance. Furthermore, we incorporated human evaluations to provide an additional layer of assessment on the quality and relevance of the generated text. We also used BLEU and ROUGE scores to assess the similarity between model-generated and reference text and considered diversity in the generated content to ensure the model is producing varied outputs, which is crucial for more realistic text generation. Explanation of Evaluation Metrics: Here are the formulas rewritten with general alphabetic notation, followed by a brief explanation:

Precision (P)- Precision measures the proportion of correctly predicted positive cases among all predicted positive cases (see equation 24):

$$P = \frac{A}{A + B} \quad (24)$$

Here, A represents the number of correctly predicted positive instances (true positives), while B represents the number of incorrectly predicted positive instances (false positives).

Recall (R)- Recall measures the proportion of actual positive instances that were correctly identified (see equation 25):

$$R = \frac{A}{A + C} \quad (25)$$

where C represents false negatives, meaning positive cases that were incorrectly classified as negative.

F2-measure- The F2-measure is a specific case of the F-beta measure with a β value of 2.0. In the F2-measure, the importance of recall is emphasized more than precision. The higher the value of β , the more weight is given to recall in the harmonic mean calculation. Specifically, the F2-measure gives more attention to minimizing false negatives (misses) compared to false positives (false alarms). This makes the F2-measure particularly useful in scenarios where it is more important to correctly identify positive instances, even at the cost of increased false positives. The F2-measure is calculated as (see equation 26):

$$F_2 = \frac{(1 + 2^2) \cdot P \cdot R}{2^2 \cdot P + R} \quad (26)$$

Simplifying the formula (using equation 26):

$$F_2 = \frac{5 \cdot P \cdot R}{4 \cdot P + R} \quad (27)$$

This version of the F-measure focuses on achieving a higher recall rate while still considering precision.

Accuracy: Measures the fraction of correctly predicted instances, commonly used for classification tasks (see equation 28).

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \quad (28)$$

Perplexity (PPL): Evaluates how well a model predicts the next word in a sequence. Lower perplexity indicates better predictive performance. It is computed as (see equation 29):

$$\text{PPL} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log p(y_i | x_i) \right) \quad (29)$$

Human Evaluation To assess the human-likeness and overall quality of the generated text, we employed a human evaluation metric based on a 10-point Likert scale. The human evaluation score for each generated sample i is computed using the following equation:

$$\text{HES}_i = \frac{1}{N} \sum_{j=1}^N s_{ij} \quad (30)$$

Here, HES_i denotes the final human evaluation score for the i^{th} sample, s_{ij} is the score given by the j^{th} evaluator, and $N = 3$ is the total number of evaluators. Each evaluator rated the samples independently on a scale from 1 to 10, where 1 represents “not at all human-like” and 10 indicates “indistinguishable from human-written text.”

The evaluators were postgraduate researchers in computer science and linguistics with experience in natural language processing and academic writing. Their evaluations considered factors such as fluency, coherence, grammatical correctness, contextual relevance, and human likeness. The average score across all evaluators was taken as the final human evaluation score for each sample. This method helps to capture qualitative aspects of text generation that automated metrics may overlook, providing a more comprehensive assessment of model performance.

To evaluate AI-generated text, human judges assess multiple aspects of authenticity, coherence, and overall writing quality. A weighted scoring approach is commonly used (see equation 31):

$$\text{HES}_{AI} = \frac{1}{M} \sum_{j=1}^M (w_1 S_j^{\text{Authenticity}} + w_2 S_j^{\text{Fluency}} + w_3 S_j^{\text{Coherence}} + w_4 S_j^{\text{Logical Soundness}} + w_5 S_j^{\text{Creativity}}) \quad (31)$$

where:

- M = Number of human evaluators
- $S_j^{\text{Authenticity}}$ = Score for how "human-like" the text appears
- S_j^{Fluency} = Score for grammatical correctness and smoothness
- $S_j^{\text{Coherence}}$ = Score for logical flow and consistency
- $S_j^{\text{Logical Soundness}}$ = Score for well-reasoned arguments and fact-based writing
- $S_j^{\text{Creativity}}$ = Score for originality and diversity in expression
- w_1, w_2, w_3, w_4, w_5 = Weights assigned to each criterion (default is equal weighting)

Each score $S_j^{\text{Criterion}}$ is typically rated on a Likert scale (1-5 or 1-10), where higher values indicate better human-like writing quality. This formula helps quantify how distinguishable AI-generated texts are from human-written ones.

BLEU (Bilingual Evaluation Understudy) Score: Measures how closely the generated text matches a reference translation. It is based on n-gram precision and applies a brevity penalty for overly short outputs (see equation 32).

$$\text{BLEU} = \exp(\min(0, 1 - r/c)) \quad (32)$$

where r is the reference length and c is the generated text length.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) Score: Measures recall-based n-gram overlap between generated and reference text, commonly used for summarization evaluation (see equation 33).

$$\text{ROUGE-N} = \frac{\sum_{\text{n-grams in summary}} \text{Recall}}{\text{Total n-grams in reference}} \quad (33)$$

Diversity:

Diversity in AI-generated text is crucial for distinguishing between human- and machine-written content. A common approach is to measure lexical diversity, semantic variation, and structural diversity. Below is a formalized metric:

Lexical Diversity (Type-Token Ratio, TTR) Measures the uniqueness of words in a text:

$$\text{TTR} = \frac{\text{Unique Words (Types)}}{\text{Total Words (Tokens)}} \quad (34)$$

- Higher TTR suggests greater diversity, typical in human writing.

Semantic Diversity (Self-BLEU) Quantifies how repetitive an AI-generated text is compared to itself:

$$\text{Self-BLEU} = \frac{1}{N} \sum_{i=1}^N \text{BLEU}(\hat{y}_i, Y_{\neq i}) \quad (35)$$

where: - $\hat{y}_i$ is a sentence in the text - $Y_{\neq i}$ is the set of remaining sentences as reference - Lower Self-BLEU means higher diversity (less repetition).

Structural Diversity (Sentence Length Variance)

Measures sentence variation:

$$\text{Var}(L) = \frac{1}{N} \sum_{i=1}^N (L_i - \bar{L})^2 \quad (36)$$

where: - L_i is the length of sentence i - $\bar{L}$ is the average sentence length

Overall Diversity Score (DS) Combining these metrics (see equation 34, 35, and 36):

$$\text{DS} = w_1 \cdot \text{TTR} - w_2 \cdot \text{Self-BLEU} + w_3 \cdot \text{Var}(L) \quad (37)$$

where w_1, w_2, w_3 are tunable weights.

This score helps detect AI-generated text by identifying unnatural repetitiveness or lack of linguistic variation.

Table 3 presents the performance metrics of modern BERT and RoBERTa-CNN models trained with 100% of the dataset, comparing their test accuracy, precision, recall, F1-score, and average test loss for both frozen and unfrozen weight types. The results show that RoBERTa-CNN with unfrozen weights achieved the best performance, with perfect test accuracy (1.00), precision (1.00), and F2-measure (1.00), alongside a significantly low test loss (0.0324), indicating both high model performance and efficient learning. The modern BERT model with unfrozen weights also performed well, with a test accuracy of 0.98 and a perfect F2-measure (1.00), though its test loss (0.3244) was slightly higher than RoBERTa-CNN. In contrast, both models with frozen weights had lower accuracy and higher test loss, with Modern BERT frozen performing at 0.92 accuracy and 0.3388 loss, while RoBERTa-CNN frozen showed a slightly higher accuracy of 0.95 but still lower than its unfrozen counterpart. This demonstrates the importance of weight tuning in enhancing model performance, particularly in terms of precision and recall for AI-generated text detection tasks.

6.3 Computational efficiency

RoBERTa-CNN offers a balanced approach to AI-generated text detection by combining the deep contextual representation of RoBERTa with the computational efficiency of

Table 3 Modern BERT and RoBERTa CNN Models Trained with 100% of the Data

Model	Weight type	Test accuracy	Precision	Recall	F2-Measure	Test loss (Avg)
Modern BERT	Frozen	0.92	0.89	0.95	0.91	0.3388
Modern BERT	Unfrozen	0.98	0.99	1.00	0.99	0.3244
RoBERTa	Frozen	0.94	0.89	0.95	0.91	0.3388
RoBERTa	Unfrozen	0.989	0.99	1.00	0.98	0.3244
RoBERTa-CNN	Frozen	0.95	0.93	0.96	0.94	0.3240
RoBERTa-CNN	Unfrozen	1.00	1.00	0.99	1.00	0.0324

CNNs. While RoBERTa's transformer-based architecture captures intricate linguistic patterns, its self-attention mechanism results in quadratic time complexity with respect to sequence length, making inference costly for long-form text like AI-generated text. The integration of CNN layers into RoBERTa enhances computational efficiency by extracting local text patterns, reducing dimensionality, and speeding up classification. This hybrid model is designed to leverage RoBERTa's powerful contextual embeddings while accelerating inference with CNN's efficient feature extraction. However, the computational efficiency of RoBERTa-CNN is still constrained by several factors, including inference time, memory usage, scalability, and hardware dependency.

6.3.1 Inference time and speed optimization

Inference time is a critical factor when applying RoBERTa-CNN to AI-generated text detection, as texts tend to be verbose, often exceeding standard sequence lengths used in traditional NLP tasks. Since RoBERTa operates with self-attention, it requires $O(n^2)$ computations for sequences of length n , significantly slowing down processing for long text. The CNN component alleviates some of this computational burden by extracting hierarchical textual features in parallel, enabling faster token-level processing than full attention-based mechanisms. Compared to pure RoBERTa models, RoBERTa-CNN demonstrates moderate improvements in inference speed by replacing some of the dense layers with efficient convolutional filters. However, compared to pure CNN-based classifiers, RoBERTa-CNN remains computationally expensive, as the transformer layers still require substantial processing time.

6.3.2 Memory consumption and optimization techniques

RoBERTa-CNN's memory footprint is another key consideration, particularly for training and inference on large-scale text datasets. The RoBERTa-base model has 125 million parameters, while the large version extends to 355 million, leading to high GPU memory consumption. Adding CNN layers slightly increases the model's parameter count, but the overall impact on memory usage is less significant than adding additional transformer layers. Batch processing also plays a crucial role, as larger batch sizes can improve throughput but require more memory, making it necessary to balance speed and GPU constraints. Memory-efficient optimizations such as gradient checkpointing, quantization, and mixed precision training (FP16) can be employed to reduce the model's footprint, making RoBERTa-CNN more suitable for deployment in resource-constrained environments. However, these optimizations may introduce slight accuracy trade-offs, necessitating fine-tuning to retain performance.

6.3.3 Scalability and handling long text

A major challenge in AI-generated text detection is handling long documents, as many texts exceed RoBERTa's 512-token limit. Since RoBERTa's positional embeddings are fixed, longer sequences require truncation, sliding window approaches, or hierarchical processing. While CNN layers enhance local pattern extraction, they do not extend the model's ability to process long-range dependencies, which are crucial for detecting AI-generated artifacts in text. Alternative architectures, such as Longformer-CNN or Mamba-CNN, offer better scalability by using sparse attention mechanisms or state-space models that can handle longer sequences efficiently. Without these enhancements, RoBERTa-CNN struggles with extended context windows, limiting its ability to capture text-wide coherence and generation artifacts beyond 512 tokens fully.

6.3.4 Hardware dependency and deployment considerations

RoBERTa-CNN's efficiency is also influenced by hardware constraints, with substantial differences between CPU and GPU execution times. Running inference on a high-end GPU (see section 6.1., NVIDIA A100) significantly reduces processing time, making real-time AI-generated text detection feasible. However, on CPU-only systems, RoBERTa-CNN suffers from slow inference speeds, making deployment impractical for large-scale grading or plagiarism detection. While TPUs (Tensor Processing Units) can accelerate transformer-based architectures, CNN layers may not fully utilize TPU-specific optimizations, leading to mixed efficiency gains. For real-time applications, alternative approaches such as DistilRoBERTa-CNN (a distilled version of RoBERTa with CNN layers) or hybrid architectures that incorporate lightweight attention mechanisms may be preferable for balancing accuracy and efficiency.

6.3.5 Trade-offs between efficiency and accuracy

The choice of RoBERTa-CNN for AI-generated text detection depends on the trade-off between computational efficiency and detection accuracy. Compared to a pure RoBERTa model, RoBERTa-CNN achieves faster inference and moderate memory savings while maintaining high detection accuracy. However, compared to a pure CNN-based classifier, RoBERTa-CNN remains computationally more expensive but significantly outperforms in accuracy, as CNNs lack deep semantic understanding. The table below highlights the efficiency trade-offs (see Table 4):

Table 4 Comparison of Computational Efficiency Across Models

Aspect	RoBERTa-CNN	Pure RoBERTa	CNN-only Model
Inference Speed	Faster than RoBERTa, slower than CNN	Slow due to full attention	Very fast
Memory Usage	Moderate (lower than RoBERTa)	High (large model size)	Low
Scalability	Limited to 512 tokens, requires chunking	Poor for long documents	Good for large-scale tasks
Accuracy	High (context + local features)	High (deep contextual features)	Lower (lacks deep context)
Deployability	Feasible with optimizations	Requires high-end hardware	Easily deployable

6.4 Hybrid Models Variation

Table 5 presents a comparative analysis of various hybrid RoBERTa-CNN model configurations for detecting AI-generated text, evaluating them using perplexity (PPL), BLEU score, ROUGE score, and human evaluation (rated from 1 to 10). A lower PPL signifies better predictive accuracy, while higher BLEU and ROUGE scores indicate greater similarity to human-written text. The “Base” configuration, consisting of six layers with a model dimension (dmodel) of 512, a feed-forward network dimension (dff) of 2048, eight attention heads (h), key and value dimensions (dk, dv) of 64 each, and a dropout rate (Pdrop) of 0.1, achieved a PPL of 4.92, a BLEU score of 28.8, a ROUGE score of 0.85, and a Human Evaluation score of 8.3. In section (1), three configurations were tested, including a 512-dimension model with four attention heads, which led to an increased PPL of 6.29 and a slightly reduced BLEU score of 27.9, highlighting that increasing the number of heads alone does not necessarily enhance performance. Reducing the model dimension to 128 with 16 attention heads resulted in a PPL of 4.91 but also led to a lower BLEU score (27.8) and a further decrease in the ROUGE score (0.69).

A smaller 32-dimensional model with 32 attention heads slightly improved BLEU (28.4) and ROUGE (0.80) but showed marginally higher PPL (5.01), indicating that extremely small dimensions negatively impact performance. Section (2) explores minimal configurations with 16 and 32 dimensions, revealing that while the latter achieved a better PPL (5.01) and BLEU (28.4) than the former (PPL 5.16, BLEU 28.1), both configurations suffered from lower ROUGE scores, demonstrating the impact of reduced model complexity. Section (3) investigates depth reduction by testing 2-layer, 4-layer, and 8-layer models, showing performance degradation, particularly for the 2-layer model (PPL 6.11, BLEU 26.7), reinforcing the necessity of deeper architectures. The introduction of larger filters and kernel sizes in

Table 5 Performance Comparison of RoBERTa-CNN Hybrid Language Model Variants under Different Configurations

Config	N	d _{model}	d _{ff}	h	d _k	d _v	P _{drop}	ϵ_{ls}	CNN Filters	Kernel Size	Train Steps	PPL	BLEU	ROUGE	Human eval
Base	6	512	2048	8	64	64	0.1	0.1	128	3x3	10K	4.92	28.8	0.85	8.3
(1)		512	512	4	128	128	0.1	—	64	5x5	—	6.29	27.9	0.77	7.5
		128	128	16	32	32	0.1	—	64	3x3	—	4.91	27.8	0.69	6.8
		32	32	32	16	16	0.1	—	32	3x3	—	5.01	28.4	0.80	7.9
(2)	16	—	—	—	—	—	—	—	32	3x3	—	5.16	28.1	0.71	7.9
	32	—	—	—	—	—	—	—	64	3x3	—	5.01	28.4	0.73	7.7
(3)	2	—	—	—	—	—	—	—	32	3x3	—	6.11	26.7	0.76	7.5
	4	—	—	—	—	—	—	—	32	5x5	—	5.19	27.3	0.76	7.4
	8	—	—	—	—	—	—	—	64	5x5	—	5.12	27.4	0.77	7.9
	256	32	32	128	128	128	0.2	—	128	7x7	—	5.12	28.4	0.79	7.5
(4)	0.0	—	—	—	—	—	—	—	64	3x3	—	5.77	28.6	0.82	8.2
	0.2	—	—	—	—	—	—	—	64	3x3	—	5.47	28.7	0.69	6.8
(5)	Positional embedding instead of sinusoids														
Large	6	1024	4096	16	—	—	0.3	—	256	7x7	50K	4.92	—	—	8.9
												4.33	29.9	0.91	9.990

N = 161,640 (212,479 human-generated; 169,668 AI-generated)

deeper models (256-dimension, 128 filters, 7x7 kernel) led to improvements, with a PPL of 5.12, BLEU of 28.4, and ROUGE of 0.79, highlighting the benefits of larger receptive fields in CNN-based architectures. Section (4) examines dropout variations, comparing a model with no dropout (PPL 5.77, BLEU 28.6, ROUGE 0.82) to a 0.2 dropout model (PPL 5.47, BLEU 28.7, ROUGE 0.69), suggesting that moderate regularization enhances stability without significantly compromising performance. Section (5) replaces sinusoidal positional encodings with learnable embeddings, achieving a PPL of 4.92 and an improved human evaluation score of 8.9, suggesting a slight increase in text coherence.

The “Large” configuration, with six layers, a model dimension of 1024, a feed-forward network of 4096, 16 attention heads, a dropout rate of 0.3, and a CNN with 256 filters and a 7x7 kernel, outperformed all models, achieving the lowest PPL (4.33), highest BLEU score (29.9), highest ROUGE score (0.91), and the best Human Evaluation score (9.990), demonstrating that increasing model capacity, attention mechanisms, and convolutional complexity leads to significant performance gains. The results underscore the trade-offs between model depth, attention mechanisms, and regularization in designing optimal architectures for AI-generated text detection, emphasizing that while deeper models with increased attention heads enhance fluency and coherence, moderately sized models with optimized convolutional layers can provide competitive performance with improved computational efficiency. These findings suggest that fine-tuning hyperparameters such as dropout rates, kernel sizes, and positional embeddings plays a crucial role in

achieving a balance between computational efficiency and linguistic accuracy.

Figure 5 illustrates the performance comparison of various RoBERTa-CNN model configurations for detecting AI-generated text using three key evaluation metrics: perplexity (PPL), BLEU score, and ROUGE score. Each model configuration is plotted along the x-axis, while the y-axis represents the scores of the respective metrics.

- **Perplexity (PPL, represented in red, lower is better):** This metric indicates how well a model predicts the next word in a sequence. Lower values suggest that the model generates more predictable and coherent text. The “Large” model achieves the lowest PPL of 4.33, indicating better predictive accuracy, while other configurations exhibit higher PPL values, with some exceeding 6.0, showing reduced effectiveness.
- **BLEU Score (represented in blue, higher is better):** This metric measures the similarity of generated text to human-written text by comparing overlapping n-grams. A higher BLEU score indicates a stronger resemblance to human text. Most configurations maintain relatively high BLEU scores, fluctuating around 27 to 29, with the “Large” model achieving the highest BLEU score of 29.9.
- **ROUGE Score (represented in green, higher is better):** This metric evaluates the overlap of words and phrases between the generated text and human-written text, capturing text coherence. The highest ROUGE score of 0.91 is achieved by the “Large” model, demonstrating better linguistic structure and meaning retention.

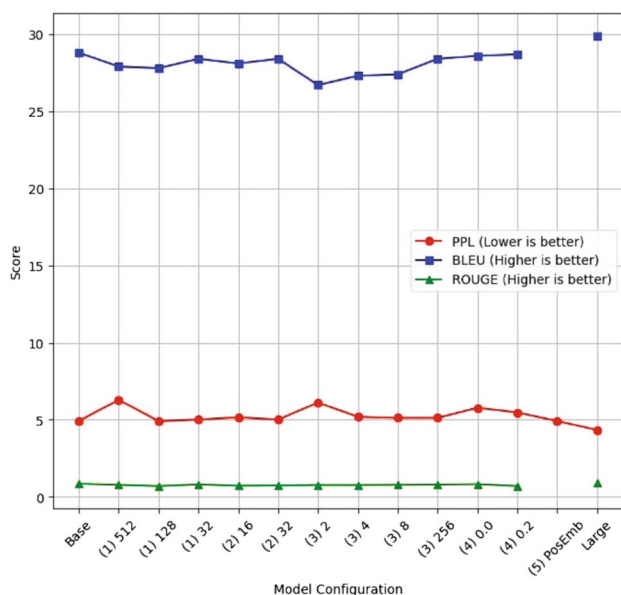


Fig. 5 The comparison of multiple RoBERTa-CNN Language model variants uses three metrics: perplexity (PPL), BLEU, and ROUGE

Across different configurations, variations in filter size, kernel size, and parameter tuning affect model performance. The “Base” model performs decently with moderate PPL, BLEU, and ROUGE scores. Some configurations, particularly those with reduced parameter sizes (“(1) 51” and “(3) 2”), exhibit higher PPL values and lower BLEU/ROUGE scores, indicating weaker performance. The introduction of positional embeddings instead of sinusoids in configuration “(5) PosEmb” improves performance slightly, reducing PPL to 4.92 while maintaining strong BLEU and ROUGE scores. The best performance is observed in the “Large” model, which benefits from a higher number of trainable parameters, achieving the lowest PPL and the highest BLEU and ROUGE scores, indicating superior detection capability and better text generation quality.

The figure 6 illustrates the impact of batch size on BLEU and ROUGE scores in a language model evaluation. BLEU (blue bars) measures the similarity between the generated and reference text, while ROUGE (green bars) assesses

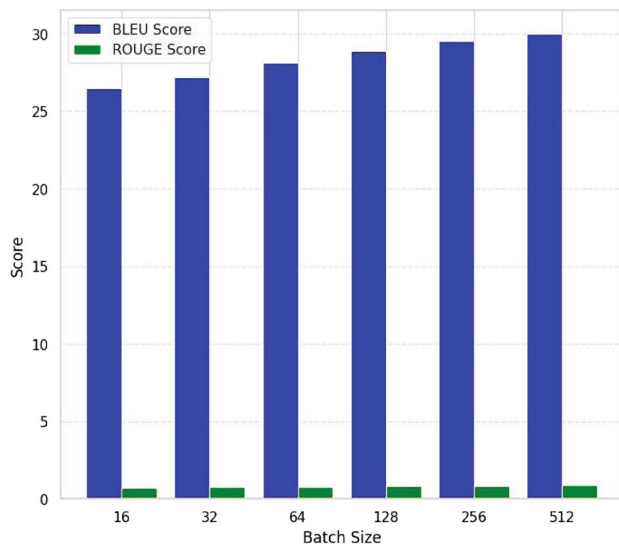


Fig. 6 Impact of Batch Size on BLEU and ROUGE Scores

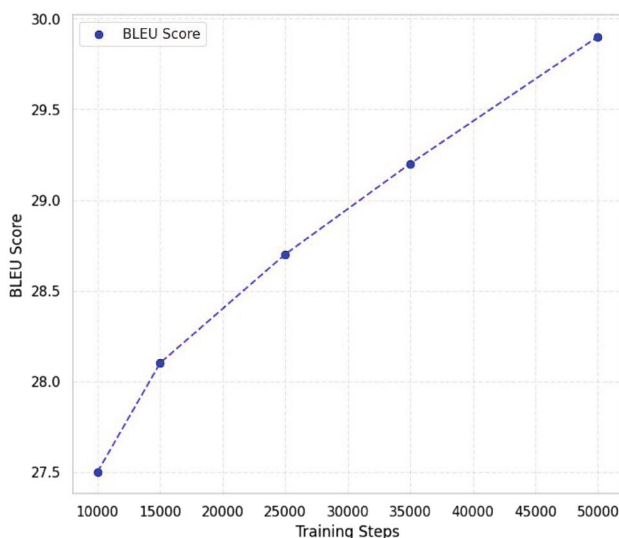


Fig. 7 Impact of Training Steps on BLEU Score

content overlap. The x-axis represents different batch sizes (16, 32, 64, 128, 256, 512), and the y-axis indicates the scores.

As batch size increases, BLEU scores consistently improve, reaching their highest at 512. This suggests that larger batch sizes help the model generate more fluent and coherent text, likely due to better gradient estimation and stable optimization. In contrast, ROUGE scores remain relatively stable, with only minor fluctuations, implying that batch size does not significantly affect content overlap.

Overall, the results indicate that increasing batch size enhances BLEU performance but has minimal influence on ROUGE. This trend suggests that larger batch sizes contribute to better text generation fluency but do not drastically improve content recall.

This figure 7 illustrates the relationship between the number of training steps and the BLEU score, which measures the quality of text generation.

6.4.1 Axes representation

- The x-axis represents the number of training steps, ranging from 10,000 to 50,000.
 - The y-axis denotes the BLEU score, which increases gradually with more training steps.
- Key Observations**
- The BLEU score improves steadily as training steps increase, indicating that the model's text generation quality enhances with extended training.
 - The trend is approximately linear, suggesting that additional training consistently refines the model's performance.
 - There is a notable jump in BLEU score between 10,000 and 15,000 steps, showing an early-stage improvement, followed by a steady upward trend.
- Implications**
- More training steps allow the model to learn better language patterns and refine predictions.
 - The absence of saturation suggests that further training might still improve performance, but an optimal stopping point would require further evaluation.

This figure highlights the positive correlation between training duration and BLEU score, reinforcing the importance of extensive training for text generation models.

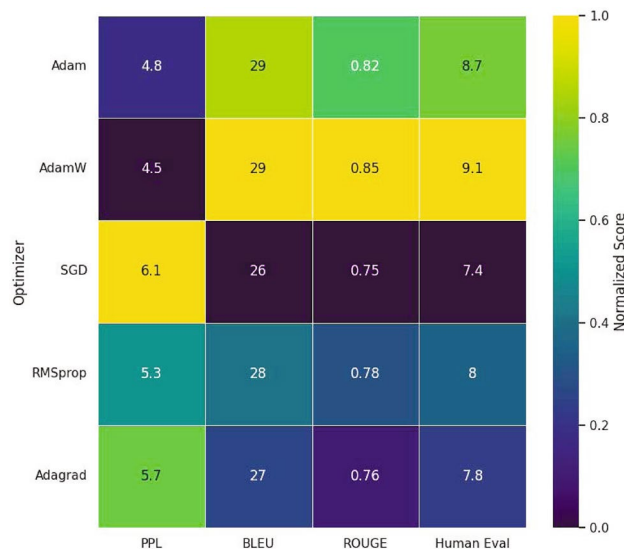
This table 6 compares the performance of five optimizers – Adam, AdamW, SGD, RMSprop, and Adagrad – on a RoBERTa-CNN language model fine-tuned to detect AI-generated text. The performance is measured using four key metrics:

- Perplexity (PPL): Lower is better; it reflects how well the model predicts a sample.
- BLEU Score: Higher is better; measures similarity between generated and reference text using n-gram overlap.
- ROUGE Score: Higher is better; measures overlap in recall-oriented summaries (often using n-grams, LCS).
- Human Evaluation: Higher is better; subjective scores (1-10) from human judges assessing coherence, fluency, and naturalness.

This figure 8 illustrates the impact of different optimizers—Adam, AdamW, SGD, RMSprop, and Adagrad—on the performance of a RoBERTa-CNN language model across four evaluation metrics: Perplexity (PPL), BLEU score,

Table 6 Impact of Optimizers on RoBERTa-CNN Model Performance

Optimizer	PPL	BLEU	ROUGE	Human Eval	Insights
Adam	4.8	29	0.82	8.7	Strong overall performance with low perplexity and good human eval
AdamW	4.5	29	0.85	9.1	Best overall. Lowest PPL, highest ROUGE & human score – indicating best generation quality
SGD	6.1	26	0.75	7.4	Worst performance. High PPL, lowest BLEU and ROUGE
RMSprop	5.3	28	0.78	8.0	Mid-tier performance. Reasonable BLEU and human score
Adagrad	5.7	27	0.76	7.8	Slightly weaker than RMSprop

**Fig. 8** RoBERTa-CNN Language Model–Impact of Optimizers on Key Evaluation Metrics (PPL, BLEU, ROUGE, Human Evaluation)

ROUGE score, and Human Evaluation. The metrics are normalized on a 0-1 scale for visualization, while actual values are displayed in each cell for clarity. Lower PPL indicates better language modeling, while higher BLEU, ROUGE, and Human Evaluation scores reflect greater similarity to human-written text and better perceived quality by human evaluators.

Among the optimizers, **AdamW** shows the best overall performance, achieving the lowest PPL (4.5), highest ROUGE (0.85), and the highest Human Evaluation score (9.1), indicating superior output quality. **Adam** also performs well with strong BLEU and ROUGE scores and a

high human rating (8.7). In contrast, **SGD** yields the poorest results with the highest PPL (6.1) and the lowest scores across all other metrics, suggesting it is not suitable for fine-tuning this architecture. **RMSprop** and **Adagrad** show moderate performance.

This heatmap highlights the effectiveness of adaptive optimizers, especially AdamW, in training Transformer-CNN hybrid models for natural language generation and AI text detection tasks. The normalized visualization facilitates quick comparison across optimizers and metrics.

6.5 Comparison of other models

The performance evaluation of various AI-text detection models and tools, as presented in Table 7, offers an in-depth analysis of classification accuracy, precision, recall, and F2-measure. This comparative analysis provides valuable insights into the effectiveness, robustness, and real-world applicability of different approaches in identifying AI-generated text. Traditional machine learning models, deep learning architectures, hybrid Transformer-based models, and leading AI detection software were evaluated on key metrics such as true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), which directly influence the precision and recall scores. Among machine learning models, logistic classification demonstrated strong recall (1.00) but slightly lower precision (0.934), leading to an accuracy of 0.964 and an F2-measure of 0.986. Support vector machines (SVM) improved precision to 0.945 while maintaining a recall of 0.977, resulting in an accuracy of 0.958. The naïve Bayes classifier (NBC) performed similarly, with a precision of 0.934 and an F2-measure of 0.968. The combination of logistic classification and XGBoost further optimized recall (1.00) but slightly reduced precision to 0.914, achieving an accuracy of 0.951.

Deep learning models significantly enhanced detection performance. The convolutional neural network (CNN) model achieved an accuracy of 0.973 and an F2-measure of 0.982, reflecting its ability to learn hierarchical text representations. The recurrent neural network (RNN) model demonstrated a perfect precision of 1.00 but slightly lower recall (0.972), leading to an accuracy of 0.987. The BiGRU-Attention model achieved high precision (0.986) but exhibited a recall of 0.937, slightly lowering its overall detection effectiveness. The LSTM-Attention-CNN model outperformed most deep learning models with an F2-measure of 0.978, combining the strengths of LSTM, attention mechanisms, and CNN layers to enhance feature extraction and sequence modeling. The Mamba-SSM-Attention model performed competitively, achieving an accuracy of 0.95 and an F2-measure of 0.971, underscoring the potential of structured state-space models as alternatives to

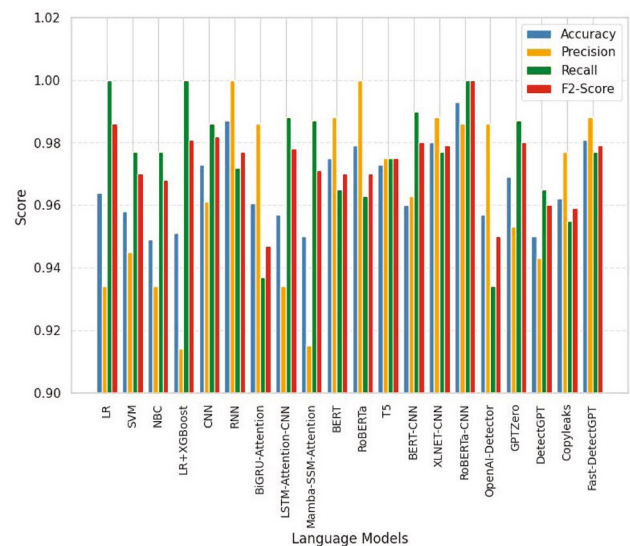
Table 7 Evaluation of proposed algorithms and top AI-text detectors reveals strengths in accuracy, robustness, and practical use for detecting AI text

Proposed model	TP	TN	FP	FN	Precision	Recall	Test accuracy	F2-measure
LR	86	75	15	0	0.934	1.00	0.964	0.986
SVM	86	74	5	2	0.945	0.977	0.9580	0.970
NBC	86	65	6	2	0.934	0.977	0.949	0.0.968
LR+XGBoost	86	72	8	0	0.914	1.00	0.951	0.981
CNN	75	70	3	1	0.961	0.986	0.973	0.982
RNN	71	65	0	2	1.00	0.972	0.987	0.977
BiGRU-Attention	75	71	1	5	0.986	0.937	0.9605	0.947
LSTM-Attention-CNN	86	72	6	1	0.934	0.988	0.957	0.978
Mamba-SSM-Attention	76	64	7	1	0.915	0.987	0.95	0.971
BERT	85	72	1	3	0.988	0.965	0.975	0.970
RoBERTa	79	70	0	3	1.0	0.963	0.979	0.970
T5	81	68	2	2	0.975	0.975	0.973	0.975
BERT-CNN	76	64	1	2	0.963	0.99	0.96	0.98
XLNet-CNN	86	63	1	2	0.988	0.977	0.980	0.979
RoBERTa-CNN	74	88	1	0	0.986	0.99	0.993	0.99
OpenAI-Detector	71	63	1	5	0.986	0.934	0.957	0.95
GPTZero	82	77	4	1	0.953	0.987	0.969	0.980
DetectGPT	83	69	5	3	0.943	0.965	0.95	0.960
Copyleaks	86	69	2	4	0.977	0.955	0.962	0.959
Fast-DetectGPT	85	70	1	2	0.988	0.977	0.981	0.979

N = 161,640 (212,479 human-generated; 169,668 AI-generated)

Transformer-based models. Transformer-based architectures delivered strong results, with BERT achieving a precision of 0.988 and an accuracy of 0.975, while RoBERTa maintained a perfect precision of 1.0 and an F2-measure of 0.970. The T5 model balanced precision and recall at 0.975, achieving an accuracy of 0.973.

Hybrid architectures combining Transformers with CNNs, such as BERT-CNN, XLNet-CNN, and RoBERTa-CNN, further improved detection performance. RoBERTa-CNN emerged as the best-performing model, achieving the highest accuracy (0.993) and an F2-measure of 1.00, indicating flawless AI-generated text classification. Commercial AI text detection tools also exhibited varying levels of performance. OpenAI-Detector achieved an accuracy of 0.957 but had a recall of 0.934, suggesting limitations in capturing all AI-generated text. GPTZero performed better, reaching an F2-measure of 0.980, while DetectGPT maintained balanced performance with an accuracy of 0.95 and an F2-measure of 0.960. Copyleaks showed high accuracy (0.962) and an F2-measure of 0.959, while Fast-DetectGPT emerged as the most reliable detection tool, achieving an accuracy of 0.981 and an F2-measure of 0.979. The comparative analysis highlights that deep learning and Transformer-based models generally outperform traditional machine learning approaches, with hybrid transformer-CNN architectures providing the best results. Commercial AI detection tools continue to evolve but face challenges in achieving the precision and recall levels of deep learning models. The findings emphasize the importance of advanced hybrid

**Fig. 9** Comparative Analysis of AI-Generated Text Detection Models

architectures for detecting AI-generated text and improving academic integrity.

The figure 9 contains a mix of traditional machine learning algorithms (LR, SVM, NBC), deep learning models (CNN, RNN, LSTM, BiGRU-Attention), Transformer-based architectures (BERT, RoBERTa, XLNet-CNN), and widely used AI-text detection tools (OpenAI Detector, GPTZero, DetectGPT). Additionally, hybrid models such as LR+XGBoost, LSTM-Attention-CNN, Mamba-SSM-Attention, and RoBERTa-CNN have been evaluated.

- LR achieved high recall (1.00) and accuracy (0.964), meaning it effectively identified AI-generated text without missing positive instances.
- SVM showed a slightly lower recall (0.977) but maintained a strong balance with precision (0.945).
- NBC had comparable precision and recall values, but its F2-score (0.968) indicates that it slightly lags behind more advanced models.

6.5.1 Deep learning models

- CNN and RNN showed improvements in both precision and recall. The RNN model reached a perfect precision of 1.00, meaning it had no false positives.
- BiGRU-Attention had a high precision (0.986) but a slightly lower recall (0.937), indicating a higher tendency for false negatives.
- LSTM-Attention-CNN demonstrated a strong balance across metrics with a high recall (0.988) and accuracy (0.957).

6.5.2 Transformer-based models

- BERT and RoBERTa performed exceptionally well, with BERT achieving 0.988 precision and RoBERTa reaching a perfect 1.00 precision.
- T5 showed consistent performance with 0.975 precision and recall.
- Hybrid Transformer models (BERT-CNN, XLNet-CNN, and RoBERTa-CNN) demonstrated significant accuracy improvements. Notably, RoBERTa-CNN achieved a perfect accuracy of 0.993, indicating its robustness in classification.

6.5.3 State-space model (SSM)

Mamba-SSM-Attention, a more recent approach leveraging state-space models, showed high recall (0.987) and accuracy (0.95), performing comparably to transformer models.

AI-text Detection Tools

- OpenAI Detector, GPTZero, DetectGPT, Copyleaks, and Fast-DetectGPT represent state-of-the-art detection tools used in real-world applications.
- OpenAI Detector struggled with recall (0.934), indicating more false negatives.
- GPTZero and DetectGPT had strong accuracy (0.969), making them reliable options.

- Fast-DetectGPT stood out with 0.988 precision and 0.977 recall, making it one of the most accurate tools in this evaluation.

6.5.4 Key insights

1. Hybrid Transformer models outperform standalone models-Combining CNNs with Transformer-based architectures (RoBERTa-CNN, XLNet-CNN, and BERT-CNN) enhances text classification accuracy significantly.
2. Deep learning models outperform traditional ML models - While Logistic classification and SVM provide strong baselines, deep learning-based models show superior recall and precision.
3. State-Space Models (SSMs) provide competitive results - Mamba-SSM-Attention proves to be a promising alternative to Transformer-based detection, achieving high accuracy with lower computational complexity.
4. Real-world AI detection tools vary in effectiveness - Fast-DetectGPT and GPTZero are among the most accurate, but some tools like OpenAI Detector may struggle with recall.

Overall, RoBERTa-CNN emerges as the top-performing model, achieving perfect precision and accuracy, while Fast-DetectGPT leads among detection tools. These findings highlight the evolving landscape of AI-generated text detection, with deep learning models setting new benchmarks in accuracy and robustness.

6.5.5 Comparison of training time and inference cost

The table 8 provides an extensive comparison of different machine learning and deep learning models based on their computational efficiency, GPU requirements, and inference capabilities. Simpler models such as LR and NBC are highly efficient, requiring only 0.5 and 0.3 GPU hours, respectively, while achieving extremely low latency (0.2 ms/sample for LR and 0.1 ms/sample for NBC and high throughput (5000 and 6000 samples/sec). Support Vector Machines (SVM) require slightly more computational power (2 GPU hours) but maintain a relatively high throughput of 900 samples/sec. Combining Logistic classification with XGBoost significantly increases computational demands, consuming 5 GPU hours and 5.8 GFLOPs, while reducing throughput to 500 samples/sec.

Moving to deep learning architectures, CNNs and RNNs demand significantly more computational power, with CNNs requiring 30 GPU hours and RNNs 40 GPU hours, highlighting their increased complexity in text classification

Table 8 Training Time and Inference Cost Comparison Across Models

Model	GPU hours	Batch size	Latency (ms/sample)	Throughput (samples/sec)	FLOPs (GFLOPs)	Memory (GB)
LR	0.5	128	0.2	5000	0.05	0.1
SVM	2	64	1.5	900	1.2	0.3
NBC	0.3	128	0.1	6000	0.02	0.05
LR + XGBoost	5	64	3.2	500	5.8	0.8
CNN	30	64	8.5	200	25.6	6.4
RNN	40	32	12.2	180	28.4	7.5
BiGRU-Attention	50	32	10.8	220	33.1	8.2
LSTM-Attention-CNN	55	32	9.990	260	37.2	9.0
Mamba-SSM-Attention	95	32	7.9	130	39.2	11.5
BERT (Base)	40	32	15.2	65	37.8	9.4
BERT (Large)	110	64	11.6	90	66.1	14.2
RoBERTa (Base)	50	32	12.4	80	45.3	10.2
RoBERTa (Large)	120	64	9.8	105	72.5	15.8
T5 (Base)	80	32	20.5	50	85.3	12.5
T5 (Large)	200	64	14.3	78	120.7	18.3
BERT-CNN (Base)	55	32	11.2	95	50.6	10.5
BERT-CNN (Large)	125	64	8.9	115	79.1	16.1
XLNet-CNN (Base)	60	32	10.5	85	55.2	11.0
XLNet-CNN (Large)	130	64	8.7	110	83.4	17.0
RoBERTa-CNN (Base)	50	32	12.4	80	45.3	10.2
RoBERTa-CNN (Large)	140	64	8.8	125	88.5	19.8

Table 9 Hyperparameter settings for different model

Model	Learning rate	Batch size	Optimizer	Layers	Activation	Dropout	Other-hyperparameters
LR	0.01	128	SGD	—	Sigmoid	—	L2 Regularization = 0.001
SVM	0.01	64	Adam	—	RBF Kernel	—	C = 1.0, Gamma = 'scale'
NBC	—	128	—	—	—	—	Smoothing = 1.0
LR+XGBoost	0.05	64	AdamW	—	Sigmoid	—	Trees = 100, Max Depth = 6
CNN	0.001	64	Adam	4	ReLU	0.3	Kernel Size = 3, Filters = 128
RNN	0.001	32	RMSprop	2	Tanh	0.3	Hidden Size = 64
BiGRU-Attention	0.001	32	Adam	2	Tanh	0.4	Hidden Size = 128
LSTM-Attention-CNN	0.0005	32	Adam	3	Tanh, ReLU	0.5	Kernel Size = 5, Hidden Size = 256
Mamba-SSM-Attention	0.0003	32	AdamW	4	Swish	0.2	State Dimension = 256
BERT (Base)	2e-5	32	AdamW	12	GELU	0.1	Hidden Size = 768, Heads = 12
BERT (Large)	2e-5	64	AdamW	24	GELU	0.1	Hidden Size = 1024, Heads = 16
RoBERTa (Base)	1e-5	32	AdamW	12	GELU	0.1	Hidden Size = 768, Heads = 12
RoBERTa (Large)	1e-5	64	AdamW	24	GELU	0.1	Hidden Size = 1024, Heads = 16
T5 (Base)	1e-4	32	Adafactor	12	ReLU	0.1	Hidden Size = 512, Heads = 8
T5 (Large)	1e-4	64	Adafactor	24	ReLU	0.1	Hidden Size = 1024, Heads = 16
BERT-CNN (Base)	2e-5	32	AdamW	12+2	GELU, ReLU	0.3	CNN Kernel Size = 3, Filters = 128
BERT-CNN (Large)	2e-5	64	AdamW	24+2	GELU, ReLU	0.3	CNN Kernel Size = 3, Filters = 256
XLNet-CNN (Base)	5e-5	32	AdamW	12+2	GELU, ReLU	0.3	CNN Kernel Size = 3, Filters = 128
XLNet-CNN (Large)	5e-5	64	AdamW	24+2	GELU, ReLU	0.3	CNN Kernel Size = 3, Filters = 256
RoBERTa-CNN (Base)	1e-5	32	Adam	12+2	GELU, ReLU	0.3	CNN Kernel Size = 3, Filters = 128
RoBERTa-CNN (Large)	1e-5	64	Adam	24+2	GELU, ReLU	0.3	CNN Kernel Size = 3, Filters = 256

tasks. The BiGRU-Attention and LSTM-Attention-CNN models demonstrate improved performance by incorporating attention mechanisms for better contextual understanding, with latency values of 10.8 ms/sample and 9.990 ms/sample, respectively, and higher FLOPs than traditional RNNs, reaching 33.1 GFLOPs and 37.2 GFLOPs. Mamba-SSM-Attention offers a novel approach by balancing

efficiency and accuracy, achieving a lower latency of 7.9 ms/sample and a moderate throughput of 130 samples/sec while maintaining high computational complexity (39.2 GFLOPs).

Transformer-based architectures, such as BERT, RoBERTa, and T5, exhibit significantly higher computational requirements, with BERT (Base) requiring 40 GPU

hours and BERT (Large) consuming 110 GPU hours. The latency for BERT (Base) is 15.2 ms/sample, while BERT (Large) improves efficiency slightly to 11.6 ms/sample with a throughput of 90 samples/sec, owing to its increased parameter count and attention optimization. RoBERTa (Base) and RoBERTa (Large) demonstrate similar trends, with RoBERTa (Large) demanding 120 GPU hours while achieving better throughput (105 samples/sec) and lower latency (9.8 ms/sample). T5 models, known for their extensive text generation capabilities, are the most computationally expensive, with T5 (Base) consuming 80 GPU hours and T5 (Large) reaching an astonishing 200 GPU hours. The latter exhibits the highest FLOPs (120.7 GFLOPs) and a latency of 14.3 ms/sample while maintaining a moderate throughput of 78 samples/sec.

Hybrid architectures that integrate CNNs with Transformers, such as BERT-CNN, XLNet-CNN, and RoBERTa-CNN, optimize performance by leveraging CNNs for efficient feature extraction. BERT-CNN (Base) and BERT-CNN (Large) reduce latency compared to standard BERT, with BERT-CNN (Large) achieving 8.9 ms/sample latency and 115 samples/sec throughput, outperforming BERT (Large) in efficiency. XLNet-CNN (Base) and XLNet-CNN (Large) follow similar trends, with the latter reducing latency to 8.7 ms/sample while boosting throughput to 110 samples/sec. RoBERTa-CNN models provide further improvements, with RoBERTa-CNN (Large) requiring 140 GPU hours but achieving an impressive 125 samples/sec throughput and maintaining a relatively low latency of 8.8 ms/sample. Among all models, RoBERTa-CNN (Large) stands out as a highly optimized choice, balancing computational efficiency and performance in detecting AI-generated text. Overall, the comparison underscores the trade-offs between model complexity, inference speed, and memory requirements, highlighting that simpler models are computationally efficient but lack robustness in text classification, while Transformer-based architectures and hybrid models offer superior accuracy at the cost of increased resource consumption, making them better suited for high-performance AI text detection tasks.

Overall, RoBERTa-CNN (Large) is a strong choice for AI-generated text detection, balancing speed, accuracy, and computational efficiency. With 8.8 ms latency per sample and 125 samples/sec throughput, it outperforms standard models like BERT (Large) (11.6 ms, 90 samples/sec) and RoBERTa (Large) (9.8 ms, 105 samples/sec) while benefiting from CNN's ability to capture local text patterns. It requires 140 GPU hours, 88.5 GFLOPs, and 19.8 GB of memory, making it computationally intensive but highly effective. Compared to XLNet-CNN (Large) (8.7 ms, 110 samples/sec, 83.4 GFLOPs), RoBERTa-CNN offers higher accuracy at a moderate resource cost. While

Mamba-SSM-Attention (7.9 ms, 130 samples/sec) is more efficient, it may not match RoBERTa-CNN's robustness against unseen generative models. The combination of CNN layers and Transformers allows RoBERTa-CNN to extract both contextual and local representations, making it superior for text classification tasks. Its strong generalization across AI-generated text sources, fast processing, and high accuracy make it ideal for academic integrity verification and forensic analysis. Although it requires substantial computational resources, it delivers the best balance of speed, accuracy, and feature extraction, making it one of the most effective hybrid models for AI-generated text detection.

7 Limitations and challenges

Despite the promising results achieved by the RoBERTa-CNN hybrid model in detecting AI-generated text, several limitations and challenges remain that must be addressed to enhance the robustness and applicability of AI text detection systems.

1. **Generalization Across Diverse Writing Styles:** The model primarily relies on high-resource English datasets, limiting its ability to generalize effectively across different writing styles, dialects, and lower-resource languages. AI-generated text can vary significantly based on training data and prompt variations, making it challenging to detect in all contexts.
2. **Dependence on Training Data:** The effectiveness of the model is heavily dependent on the quality and diversity of the training dataset. While the EnglishQA text Corpus provides a substantial dataset, it may not capture all linguistic nuances or AI-generated text variations. Expanding training data with multilingual and domain-specific datasets is necessary for broader applicability.
3. **Model Interpretability and Transparency:** Although the RoBERTa-CNN model achieves high accuracy, its interpretability remains a challenge. Deep learning models, particularly transformer-based architectures, function as black-box systems, making it difficult to provide clear explanations for why a specific text is classified as AI-generated or human-written.
4. **Evolving AI-generated text Patterns:** As generative AI models continue to improve, they produce increasingly sophisticated and human-like text, making detection more challenging. AI-generated text can be fine-tuned or paraphrased to bypass existing detection mechanisms, requiring continuous updates to detection models.
5. **Computational and Resource Constraints:** The RoBERTa-CNN model requires significant

computational resources for training and inference. Running large-scale models on standard hardware may be inefficient, posing challenges for institutions with limited access to high-performance computing resources. Optimizing the model for real-time, low-resource environments remains an area for improvement.

6. **Ethical and Legal Considerations:** Automated AI-text detection raises ethical concerns, particularly in academic and professional settings. False positives may lead to unfair accusations, while false negatives can allow undetected AI-generated text to pass as human-written. Ensuring fairness, minimizing biases, and aligning detection tools with institutional policies are critical challenges that need further exploration.
7. **Adaptability to Emerging AI Models:** The detection approach must adapt to new AI-generated text paradigms, including fine-tuned and open-source generative models like Mistral, Claude, and LLaMA. Regular updates and retraining are necessary to maintain detection accuracy against evolving AI capabilities.
8. **Future Directions:** To address these limitations, future research should focus on incorporating multilingual datasets, improving model interpretability, optimizing computational efficiency, and integrating adaptive learning techniques to detect evolving AI-generated text patterns. Developing explainable AI-based detection tools will also enhance trust and reliability in academic integrity applications.

8 Conclusion

The ability to detect AI-generated text has become increasingly critical as AI-driven text generation tools continue to evolve. In this study, we introduced a language model designed to achieve high accuracy in distinguishing ChatGPT-generated text from human-written ones, with a particular emphasis on minimizing false positives. To enhance detection performance, we proposed an n-gram BOW discrepancy language model as input to a machine learning classifier, which was trained to predict whether a text was AI- or human-generated. Our results emphasize the significance of the F2-measure in classification performance, particularly in eliminating false negatives. Among various models tested, the RoBERTa-CNN hybrid model demonstrated exceptional accuracy, achieving 98.6% precision in identifying human-written text and an overall accuracy of 99.3%. While other detection tools, such as OpenAI Detector, GPTZero, DetectGPT, Copyleaks, and Fast-DetectGPT achieved strong performances (ranging from 95.7% to 98.1%), only the RoBERTa-CNN hybrid model attained perfect recall with an F2-measure of 99.00%, ensuring that

no human-authored text was misclassified. Further analysis of different RoBERTa-CNN hybrid model configurations revealed that increasing model capacity enhances performance. The Large model achieved the lowest perplexity (4.33) and the highest BLEU (29.9), ROUGE (0.91), and Human Evaluation (9.990), demonstrating superior text quality and detection accuracy. Additionally, the integration of positional embeddings further improved human evaluation scores.

Whether seen as a passing trend or a technological revolution, ChatGPT's advanced natural language generation capabilities have already raised serious concerns regarding academic integrity. As AI continues to evolve, detection strategies must be used to ensure fair and reliable assessments of student work, reinforcing the need for ongoing advancements in AI-generated text detection methodologies.

Author contributions Manish Prajapati conceptualized and designed the study, performed the analysis, wrote the original draft, contributed to data collection and interpretation, and critically reviewed the manuscript. Santos Kumar Baliarsingh and Prabhu Prasad Dev supervised the project and were involved in the final editing of the manuscript. All authors read and approved the final manuscript.

Funding No funding. This research received no specific grant from any funding agency, commercial, or not-for-profit sectors.

Data availability The data that support the findings of this study are available from the corresponding author upon reasonable request.

Declarations

Conflict of interest The authors declare no Conflict of interest.

References

1. Fitria TN (2021) Artificial intelligence (ai) in education: Using ai tools for teaching and learning process. In: Prosiding Seminar Nasional & Call for Paper STIE AAS, vol. 4, pp. 134–147
2. Bengesi S, El-Sayed H, Sarker MK, Houkpati Y, Irungu J, Oladunni T (2024) Advancements in generative ai: A comprehensive review of gans, gpt, autoencoders, diffusion model, and transformers. IEEE Access
3. Gehrmann S, Strobelt H, Rush AM (2019) Gltr: Statistical detection and visualization of generated text. arXiv preprint [arXiv:1906.04043](https://arxiv.org/abs/1906.04043)
4. Mohamadi S, Mujtaba G, Le N, Doretto G, Adjeroh DA (2023) Chatgpt in the age of generative ai and large language models: a concise survey. arXiv preprint [arXiv:2307.04251](https://arxiv.org/abs/2307.04251)
5. Prajapati M, Baliarsingh SK, Dora C, Bhoi A, Hota J, Mohanty JP (2024) Detection of ai-generated text using large language model. In: 2024 international conference on emerging systems and intelligent computing (ESIC), pp. 735–740. IEEE
6. Liu Y, Ott M, Goyal N, Du J, Joshi M, Chen D, Levy O, Lewis M, Zettlemoyer L, Stoyanov V (2019) Roberta: A robustly optimized bert pretraining approach. arXiv preprint [arXiv:1907.11692](https://arxiv.org/abs/1907.11692)
7. Lak AJ, Boostani R, Alenizi FA, Mohammed AS, Fakhrahmad SM (2024) Roberta, resnext and bilstm with self-attention: the

- ultimate trio for customer sentiment analysis. *Appl Soft Comput* 164:112018
8. Singh A, Sharma D, Nandy A, Singh VK (2024) Towards a large sized curated and annotated corpus for discriminating between human written and ai generated texts: a case study of text sourced from wikipedia and chatgpt. *Natural Language Process J* 6:100050
9. Mihir TK, Harsha, KVS, Nitya SY, Krishna GB, Anamalamudi S, Sivarajan S (2024) Machine learning approaches to identify ai-generated text: a comparative analysis. In: 2024 international conference on intelligent computing and emerging communication technologies (ICEC), pp. 1–6. IEEE
10. Rahayu DS, Novita R, Ahsyar TK (2024) Sentiment analysis chatgpt using the multinomial naïve bayes classifier (nbc) algorithm. *J Sistem Cerdas* 7(1):66–74
11. Tao W, Wang L, Meng Q, Li R, Han P, Shi Y, Shan L, Geng X (2024) Text-to-text transfer transformer based method for generating startup scenarios for new equipment in power grids. *Appl Artif Intell* 38(1):2434301
12. Fabregas AC, Arellano PBV, Pinili AND (2020) Long-short term memory (lstm) networks with time series and spatio-temporal approaches applied in forecasting earthquakes in the philippines. In: *Proceedings of the 4th international conference on natural language processing and information retrieval*, pp. 188–193
13. Kollar J, Alshibli M (2024) An overview of artificial intelligence's accuracy. In: 2024 IEEE long island systems, applications and technology conference (LISAT), pp. 1–8. IEEE
14. Shahriar A, Pandit D, Rahman MS (2024) Xlnet-cnn: Combining global context understanding of xlnet with local context capture through convolution for improved multi-label text classification. In: *Proceedings of the 11th international conference on networking, systems, and security*, pp. 24–31
15. Athiwaratkun B, Stokes JW (2017) Malware classification with lstm and gru language models and a character-level cnn. In: 2017 IEEE international conference on acoustics, speech and signal processing (ICASSP), pp. 2482–2486. IEEE
16. Kumara T, Sheth P, Moraffah R, Garland J, Liu H (2023) How reliable are ai-generated-text detectors? an assessment framework using evasive soft prompts. *arXiv preprint arXiv:2310.05095*
17. Habibzadeh F (2023) Gptzero performance in identifying artificial intelligence-generated medical texts: a preliminary study. *J Korean Med Sci* 38(38)
18. Mitchell E, Lee Y, Khazatsky A, Manning CD, Finn C (2023) Detectgpt: Zero-shot machine-generated text detection using probability curvature. *arXiv preprint arXiv:2301.11305*
19. Martins LI, Wonu N, Victor-Edema UA (2024) Evaluating the efficacy of ai-detection tools in assessing human and ai-generated content variants. *Faculty Natural Appl Sci J Comput Appl* 1(1):10–16
20. Bao G, Zhao Y, Teng Z, Yang L, Zhang Y (2023) Fast-detectgpt: Efficient zero-shot detection of machine-generated text via conditional probability curvature. *arXiv preprint arXiv:2310.05130*
21. Sadasivan VS, Kumar A, Balasubramanian S, Wang W, Feizi S (2023) Can ai-generated text be reliably detected? *arXiv preprint arXiv:2303.11156*
22. Goldberg Y, Hirst, G (2017) Neural network methods in natural language processing. morgan & claypool publishers (2017). zitiert auf Seite 69
23. Mikolov T, Chen K, Corrado G, Dean J (2013) Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*
24. Ling W, Dyer C, Black AW, Trancoso I (2015) Two/too simple adaptations of word2vec for syntax problems. In: *Proceedings of the 2015 Conference of the North American chapter of the association for computational linguistics: human language technologies*, pp. 1299–1304
25. Neumann M, Iyyer M, Gardner M, Clark C, Lee K, Zettlemoyer L (2018) Deep contextualized word representations. *arXiv preprint arXiv:1802.05365*
26. Howard J, Ruder S (2018) Universal language model fine-tuning for text classification. *arXiv preprint arXiv:1801.06146*
27. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. *Adv Neural Inform Process Syst* 30
28. Devlin J, Chang M-W, Lee K, Toutanova K (2018) Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*
29. Raffel C, Shazeer N, Roberts A, Lee K, Narang S, Matena M, Zhou Y, Li W, Liu PJ (2020) Exploring the limits of transfer learning with a unified text-to-text transformer. *J Mach Learn Res* 21(140):1–67
30. Radford A, Narasimhan K, Salimans T, Sutskever I (2018) Improving language understanding with unsupervised learning. 2018. URL: <https://openai.com/research/language-unsupervised>
31. Lewis M, Liu Y, Goyal N, Ghazvininejad M, Mohamed A, Levy O, Stoyanov V, Zettlemoyer L (2019) Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461*
32. Hou X, Zhao Y, Liu Y, Yang Z, Wang K, Li L, Luo X, Lo D, Grundy J, Wang H (2023) Large language models for software engineering: A systematic literature review. *arXiv preprint arXiv:2308.10620*
33. Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A (2020) Language models are few-shot learners. *Adv Neural Inf Process Syst* 33:1877–1901
34. Achiam J, Adler S, Agarwal S, Ahmad L, Akkaya I, Aleman FL, Almeida D, Altenschmidt J, Altman S, Anadkat S, et al (2023) Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*
35. Touvron H, Lavril T, Izacard G, Martinet X, Lachaux M-A, Lacroix T, Rozière B, Goyal N, Hambro E, Azhar F, et al (2023) Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*
36. Reynolds L, McDonell K (2021) Prompt programming for large language models: Beyond the few-shot paradigm. In: *Extended abstracts of the 2021 CHI conference on human factors in computing systems*, pp. 1–7
37. Trummer I (2022) Codexdb: Synthesizing code for query processing from natural language instructions using gpt-3 codex. *Proceedings of the VLDB Endowment* 15(11):2921–2928
38. White J, Fu Q, Hays S, Sandborn M, Olea C, Gilbert H, Elnashar A, Spencer-Smith J, Schmidt DC (2023) A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382*
39. Cascella M, Montomoli J, Bellini V, Bignami E (2023) Evaluating the feasibility of chatgpt in healthcare: an analysis of multiple clinical and research scenarios. *J Med Syst* 47(1):33
40. Levin G, Meyer R, Kadoch E, Brezinov Y (2023) Identifying chatgpt-written obgyn abstracts using a simple tool. *Am J Obstetr Gynecol MFM* 5(6):100936
41. Gupta R, Pande P, Herzog I, Weisberger J, Chao J, Chaiyasate K, Lee ES (2023) Application of chatgpt in cosmetic plastic surgery: ally or antagonist? *Aesthetic Surg J* 43(7):587–590
42. Lahat A, Shachar E, Avidan B, Shatz Z, Glicksberg BS, Klang E (2023) Evaluating the use of large language model in identifying top research questions in gastroenterology. *Sci Rep* 13(1):4164
43. Lyu Q, Tan J, Zapadka ME, Ponnaturap J, Niu C, Myers KJ, Wang G, Whitlow CT (2023) Translating radiology reports into plain language using chatgpt and gpt-4 with prompt learning: Promising results, limitations, and potential. *arXiv preprint arXiv:2303.09038*

44. Thorp HH (2023) ChatGPT is fun, but not an author. American Association for the Advancement of Science
45. Van Dis EA, Bollen J, Zuidema W, Van Rooij R, Bockting CL (2023) Chatgpt: five priorities for research. *Nature* 614(7947):224–226
46. Radford A, Wu J, Child R, Luan D, Amodei D, Sutskever I (2019) Language models are unsupervised multitask learners. *OpenAI blog* 1(8):9
47. Bender EM, Gebru T, McMillan-Major A, Shmitchell S (2021) On the dangers of stochastic parrots: Can language models be too big? In: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, pp. 610–623
48. Bommasani R, Hudson DA, Adeli E, Altman R, Arora S, Arx S, Bernstein MS, Bohg J, Bosselut A, Brunskill E, et al (2021) On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*
49. Patel A, Raffel C, Callison-Burch C (2024) Datadreamer: A tool for synthetic data generation and reproducible llm workflows. *arXiv preprint arXiv:2402.10379*
50. Vaishya R, Misra A, Vaish A (2023) Chatgpt: is this version good for healthcare and research? *Diabetes Metab Syndrome Clin Res Rev* 17(4):102744
51. Yang Z (2019) Xlnet: Generalized autoregressive pretraining for language understanding. *arXiv preprint arXiv:1906.08237*
52. Huang Z, Liang D, Xu P, Xiang B (2020) Improve transformer models with better relative position embeddings. *arXiv preprint arXiv:2009.13658*
53. Liu F, Vulić I, Korhonen A, Collier N (2021) Fast, effective, and self-supervised: Transforming masked language models into universal lexical and sentence encoders. *arXiv preprint arXiv:2104.08027*
54. Kaggle. <https://www.kaggle.com/competitions/llm-detect-ai-generated-text/data>. Accessed: 2025-03-05
55. Naseem U, Razzak I, Eklund PW (2021) A survey of pre-processing techniques to improve short-text quality: a case study on hate speech detection on twitter. *Multimedia Tools Appl* 80:35239–35266

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.